

Module 2

2	Database Models and SQL	
	2.1	Database design phases, E-R Model, Constraints, E-R Diagrams, E-R design issues
	2.2	Entity set, Extended E-R features
	2.3	Relational model concepts, Constraints Relational Algebra: Unary, Binary and Set theory relational operations Data definition commands: attribute constraints, SET operations, Aggregate functions, Null Values, Nested sub queries, complex queries, Views Data control commands Data manipulation commands: Insert, Update, Joined relations Integrity and security: Domain constraints, Referential integrity, Triggers; Security and Authorization in SQL

Overview of Database Design Process

Two main activities:

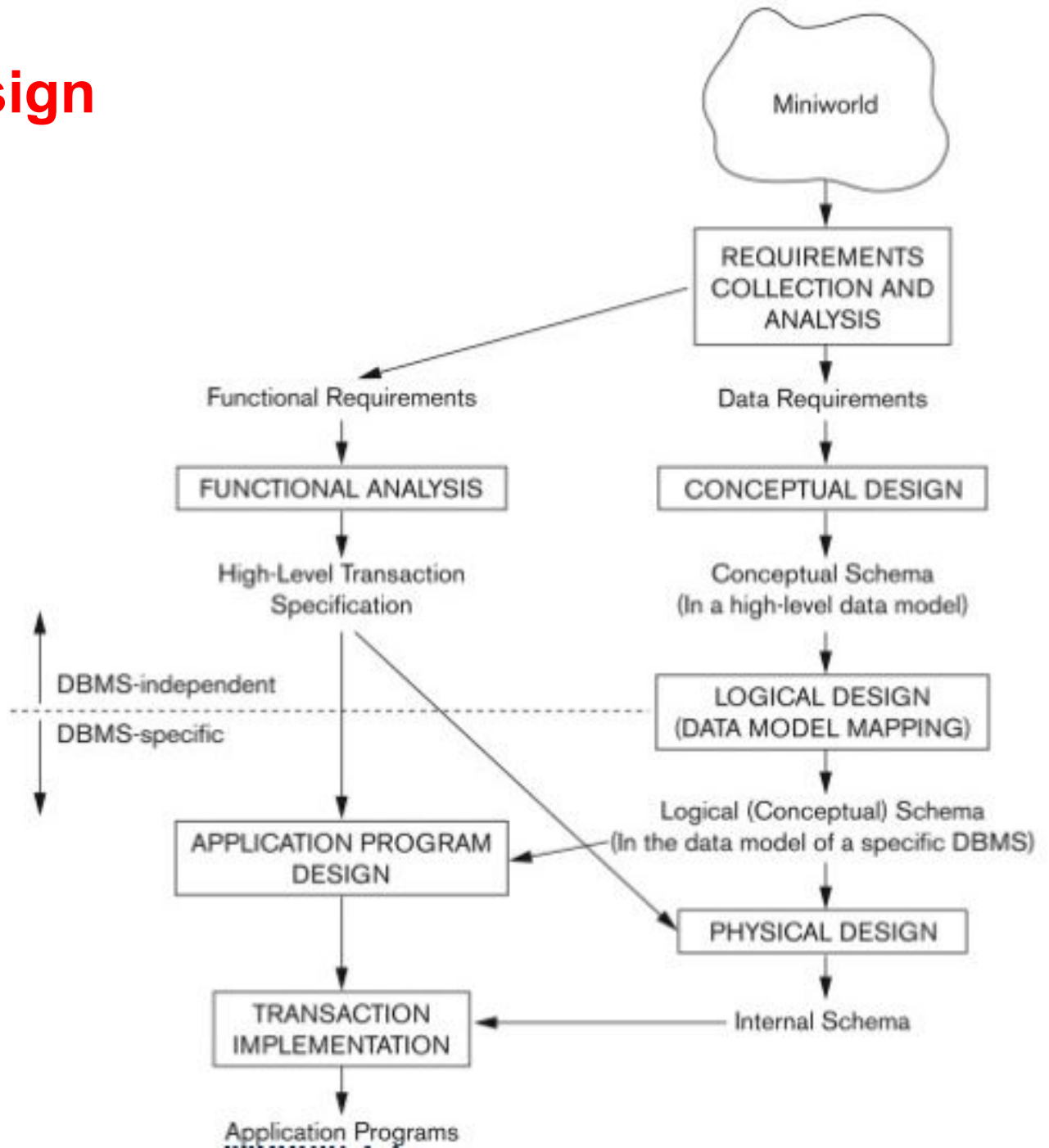
- Database design:

To design the conceptual schema for a database application

- Applications design:

To focus on the programs and interfaces that access the database

Database design phases



Database design phases

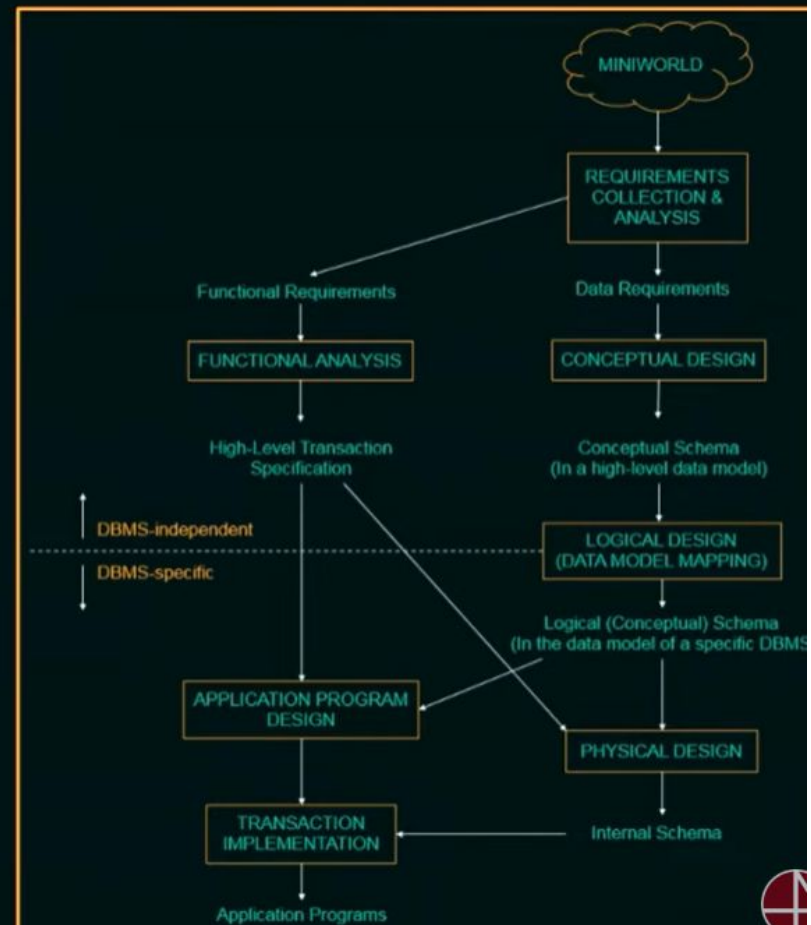
❖ Database Design

★ Requirements Collection & Analysis:

- Database designers understand & document the data requirements of the database users.

★ Functional Requirements:

- Consists of user-defined operations.



Database design phases

★ Conceptual Design:

- Creating conceptual schema.
- Conceptual Schema →
Concise description of the data requirements & detailed description of the entity types, relationships & constraints.

❖ Database Design

★ Logical Design:

- Actual implementation of the database, using commercial DBMS.

★ Physical Design:

- The internal storage structures, indexes, access paths → specified.

Database design phases

- In a **specification of functional requirements**, users describe the kinds of operations (or transactions) that will be performed on the data. **Example operations include modifying or updating data, searching for and retrieving specific data, and deleting data.** At this stage of conceptual design, the designer can review the schema to ensure that it meets functional requirements.
- In the **logical-design phase**, the designer maps the high-level conceptual schema onto the implementation data model of the database system that will be used.
- Finally, the designer uses the resulting system-specific database schema in the subsequent **physical-design phase**, in which the physical features of the database are specified.

Benefits of Data Modelling

1. Focusing on essentials:

develop simple and abstract view of process

ignore details distracting essential features

e.g. In library the fact that books have several editions may be ignored

2. Ease of communication and understanding:

communication among all parties involved to understand and document what is being modelling

e.g. In library database system communication between database

designer, librarians and users(students, professors)

Benefits of Data Modelling

3. Product or process improvement:

Communication among various stakeholders to improve process

4. Exploring alternatives:

build prototype can assist in exploring and evaluating alternatives

e.g allowing students to enroll online in university if is not been done earlier

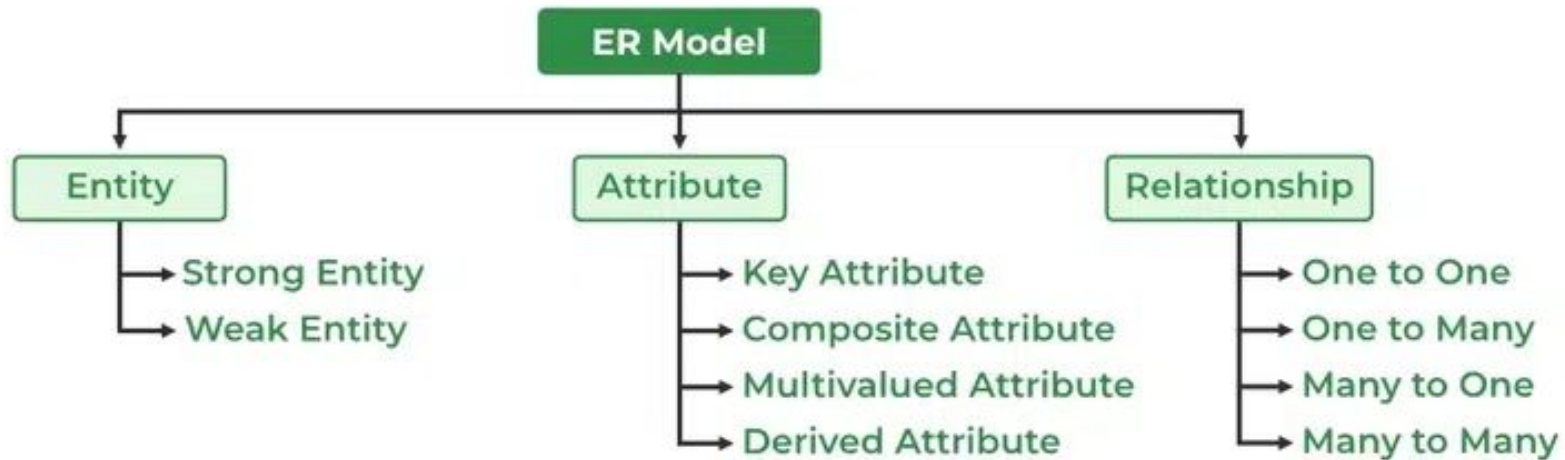
Data Modeling Using the Entity-Relationship Model

Entity-Relationship(ER) Model

The ER model

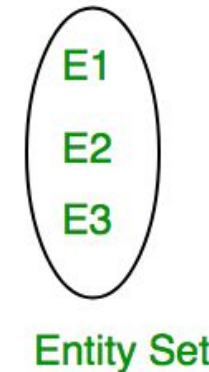
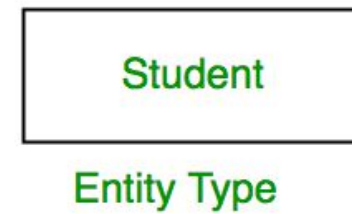
- a high-level conceptual data model
- not been implemented in any commercial DBMS
- but is a powerful short hand often used in database design for a first rendition of the mini world.
- The ER model was introduced by Peter Chen in 1976, and is now the most widely used conceptual data model.

Components of ER Model



- **Entity**

- An Entity may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence – a company, a job, or a university course.



- **Entity Set:** An Entity is an object of Entity Type and a set of all entities is called an entity set. For Example, E1 is an entity having Entity Type Student and the set of all students is called Entity Set.





Entity Name

Entity

Person, place, object, event or concept about which data is to be maintained

Example: Car, Student



Attribute Name

Attribute

Property or characteristic of an entity

Example: Color of car Entity Name of Student Entity



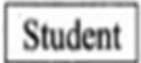
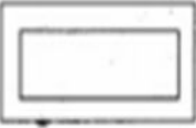
Relation

Verb Phrase

Association between the instances of one or more entity types

Example: Blue Car Belongs to Student Jack

Symbols Used in ER Model

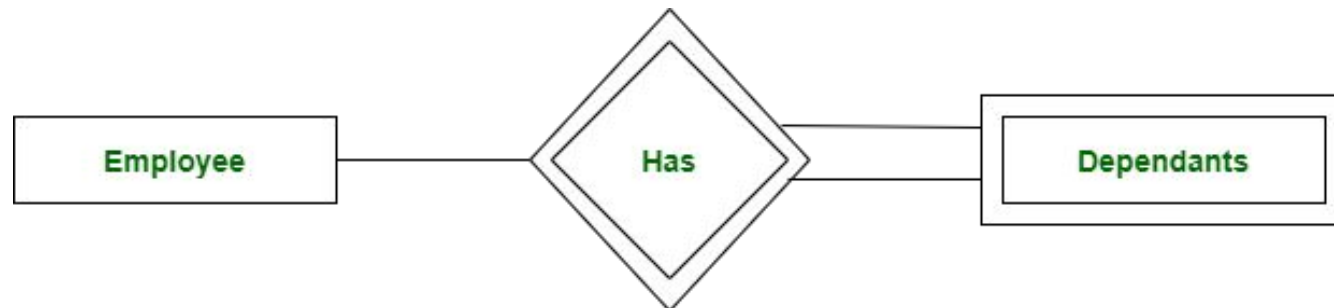
ER Component	Description (how it is represented)	Notation
Entity – Strong	Simple rectangular box	
Entity – Weak	Double rectangular boxes	
Relationships	Rhombus symbol - Strong	
between Entities	Rhombus within rhombus – Weak	
Attributes	Ellipse Symbol connected to the entity	
Key Attribute for Entity	Underline the attribute name inside Ellipse	
Derived Attribute for	Dotted ellipse inside main ellipse Entity	
Multivalued Attribute	Double Ellipse for Entity	

- **1. Strong Entity**

- A Strong Entity is a type of entity that has a key Attribute. Strong Entity does not depend on other Entity in the Schema. It has a primary key, that helps in identifying it uniquely, and it is represented by a rectangle. These are called Strong Entity Types.

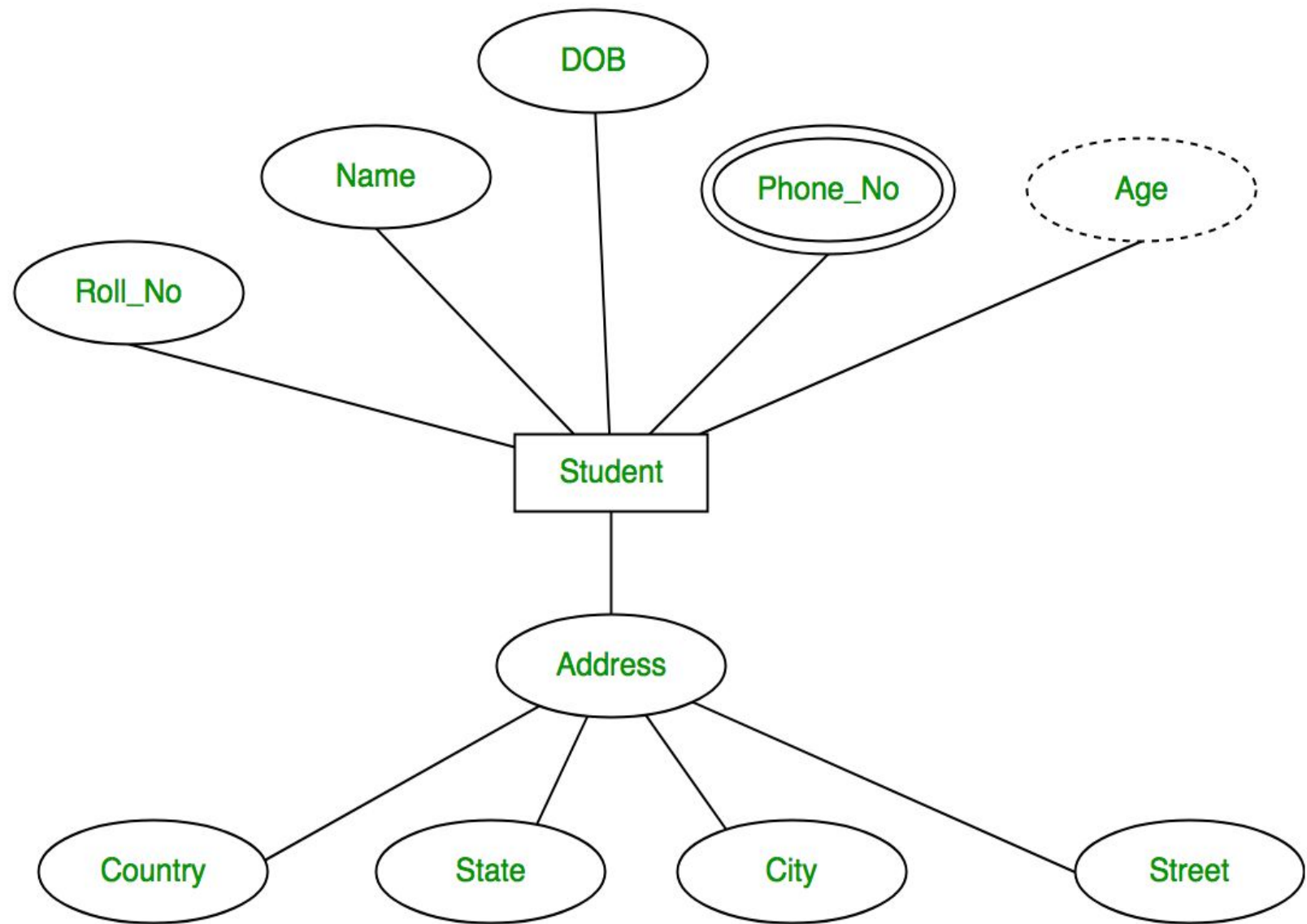
- **2. Weak Entity**

- An Entity type has a key attribute that uniquely identifies each entity in the entity set. But some entity type exists for which key attributes can't be defined. These are called Weak Entity types.
- **For Example**, A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents don't have existed without the employee. So Dependent will be a **Weak Entity Type** and Employee will be Identifying Entity type for Dependent, which means it is **Strong Entity Type**.



Attributes

- Attributes are the properties that define the entity type. For example, Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student.
- **Key Attribute**
 - The attribute which **uniquely identifies each entity** in the entity set is called the key attribute. For example, Roll_No will be unique for each student.
- **Composite Attribute**
 - An attribute **composed of many other attributes** is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.
- **Multivalued Attribute**
 - An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student).
- **Derived Attribute**
 - An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB).



Example COMPANY Database

- We need to create a database schema design based on the following (simplified) requirements of the COMPANY Database:
 - The company is organized into DEPARTMENTS. Each department has a **name**, **number** and an **employee** who manages the department.
 - We keep track of the **start date** of the department manager. A department **may have several locations**.
 - Each department controls a **number of PROJECTs**. Each project has a unique name, unique number and is located at a single location.

Example COMPANY Database

- We store each EMPLOYEE's social security number, address, salary, gender, and birthdate.
 - Each employee works for one department but may work on several projects.
 - We keep track of the number of hours per week that an employee currently works on each project.
- We also keep track of the direct supervisor of each employee.
 - Each employee may have a number of DEPENDENTS.
 - For each dependent, we keep track of their name, gender, birthdate, and relationship to the employee.

ER Model Concepts

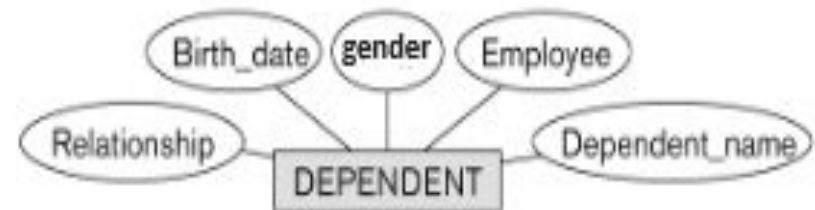
- **Entities and Attributes**

- **Entities(noun)** are specific objects or things in the mini-world that are represented in the database.
 - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
- **Attributes** are properties used to describe an entity.
 - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, gender, BirthDate.
- A specific entity will have a value for each of its attributes.
 - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', gender='M', BirthDate='09-JAN-55'
- Each attribute has a value set (or data type) associated with it – e.g. integer, string, subrange, enumerated type, ...

Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
 - DEPARTMENT
 - PROJECT
 - EMPLOYEE
 - DEPENDENT
- The initial attributes shown are derived from the requirements description

Initial Design of Entity Types:



Structural Constraints on Relationships

Participation constraint:

- Total participation
- Partial participation

Mapping Cardinality constraints:

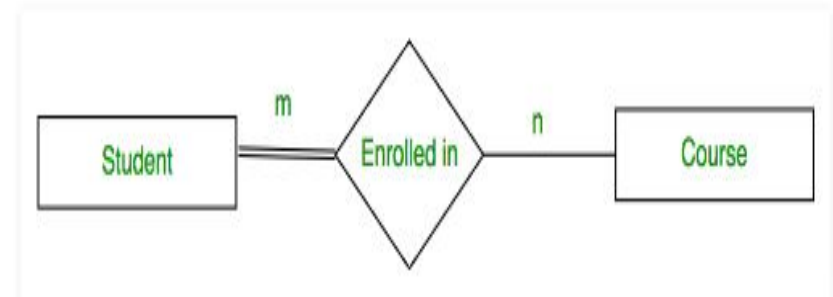
- One-to-one (1:1)
- One-to-many (1:N)
- Many-to-one (N:1)
- Many-to-many

Participation Constraint

- Participation Constraint is applied on the entity participating in the relationship set.

1) Total Participation:

- Each entity in the entity set **must participate** in the relationship.
- If each student **must** enroll in a course, the participation of student will be total.
- Total participation is shown by double line in ER diagram.



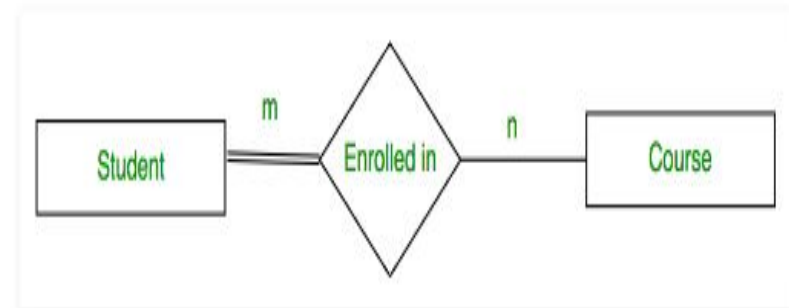
Types of Entity Sets

Symbols Used-

- A single rectangle is used for representing a strong entity set.
- A diamond symbol is used for representing the relationship that exists between two strong entity sets.
- A single line is used for representing the partial participation of the strong entity set with the relationship set.
- A double line is used for representing the total participation of an entity set with the relationship set.
- Total participation may or may not exist in the relationship.

2. Partial Participation

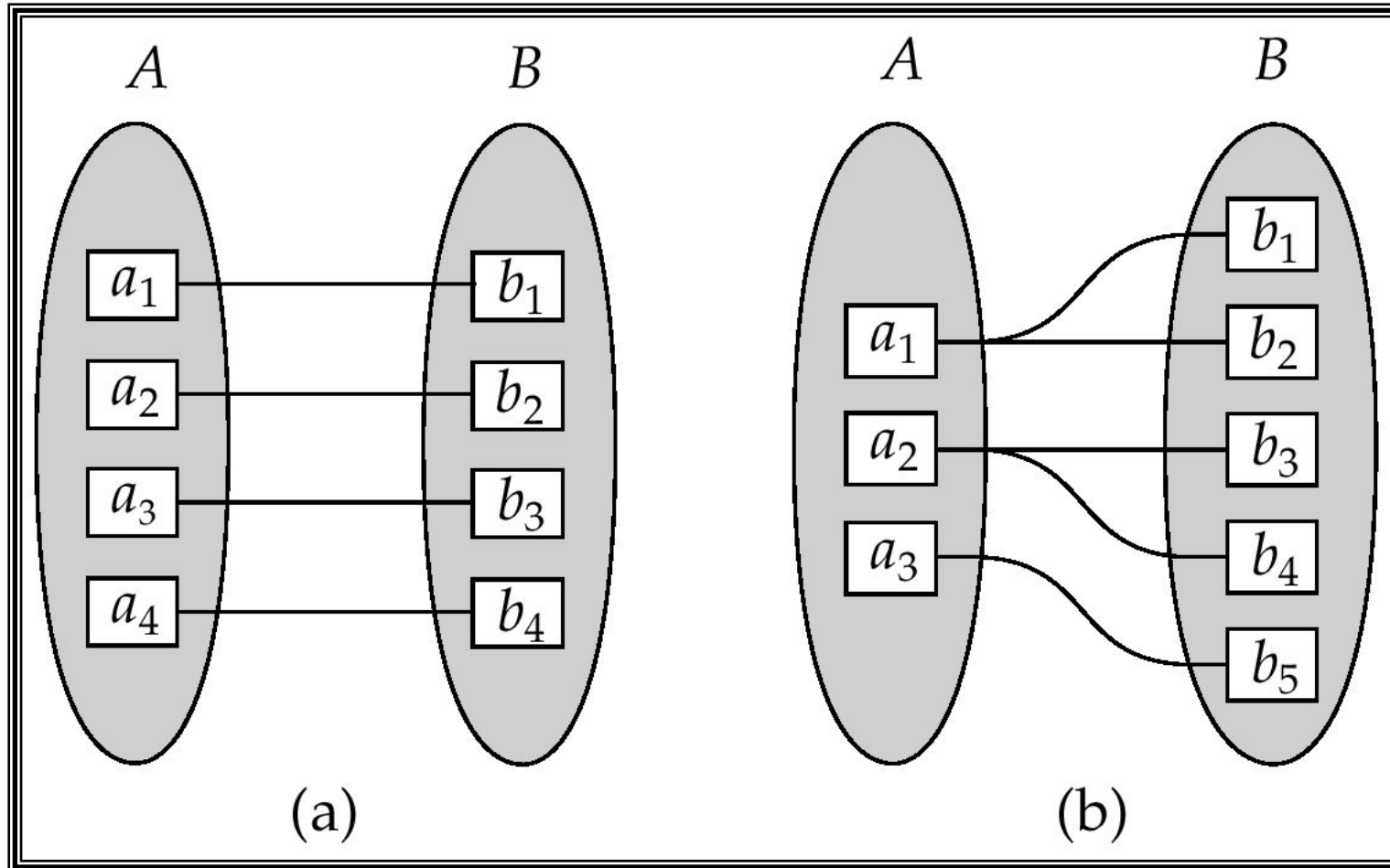
- The entity in the entity set may or may NOT participate in the relationship.
- If some courses are not enrolled by any of the student, the participation of course will be partial.
- The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Mapping Cardinality/Mapping Constraints

- Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.
- For a binary relationship set R between entity sets A and B , the mapping **cardinality must be** one of the following:
 - One-To-One
 - One-To-Many
 - Many-To-One
 - Many-To-Many

1.Mapping cardinality Constraints



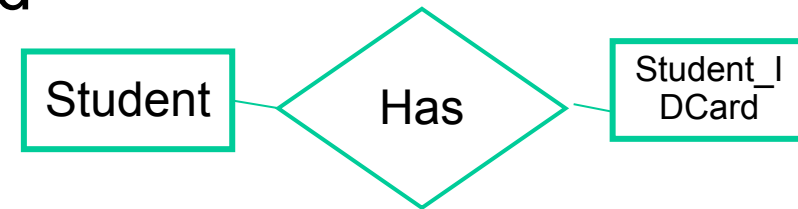
One to one

One to many

Mapping Constraints

1.One-To-One

- An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.
- E.g. In a **university database**, a **student** can have **one ID card**, and each **ID card** is assigned to **one student**.
- We draw a directed line from the relationship set advisor to both entity sets instructor and student.
- This indicates that an instructor may advise Each student has exactly one ID card, and each ID card belongs to exactly one student.



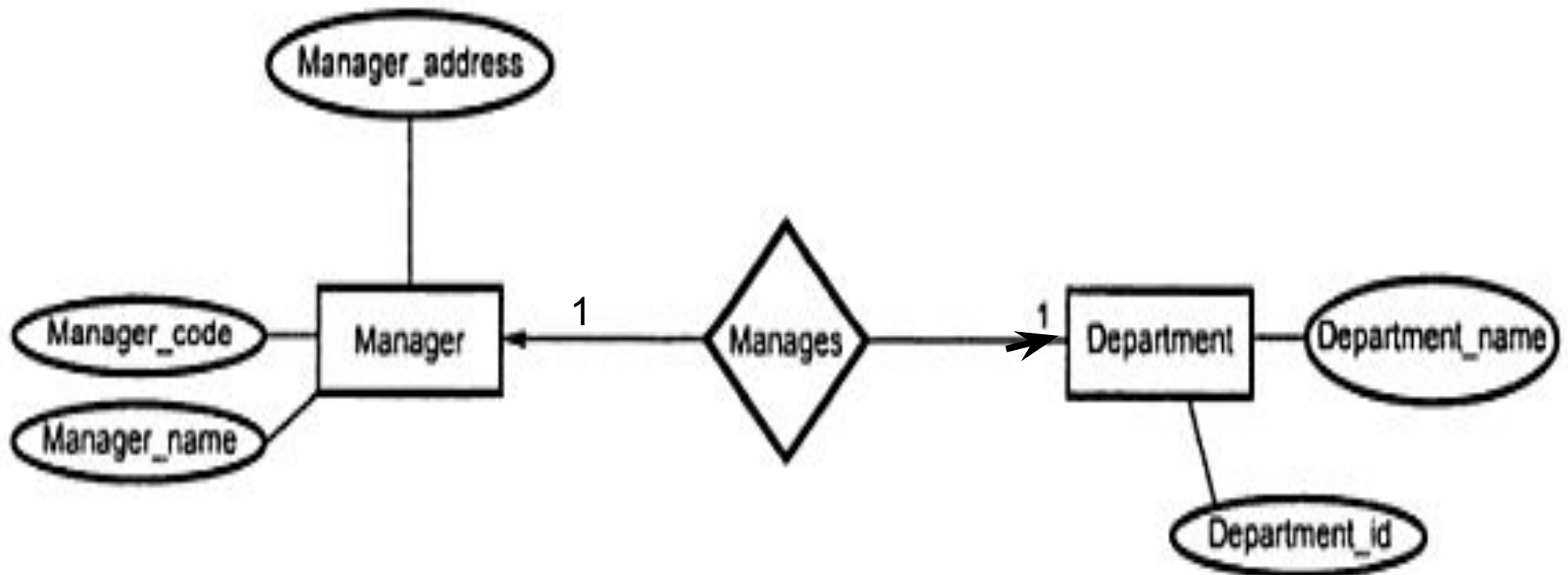
(a) One-to-one

Mapping Constraints

1.One-To-One

Constraints

Mapping cardinality Constraints 1 to 1



Mapping Cardinality

2.One-To-Many

An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.

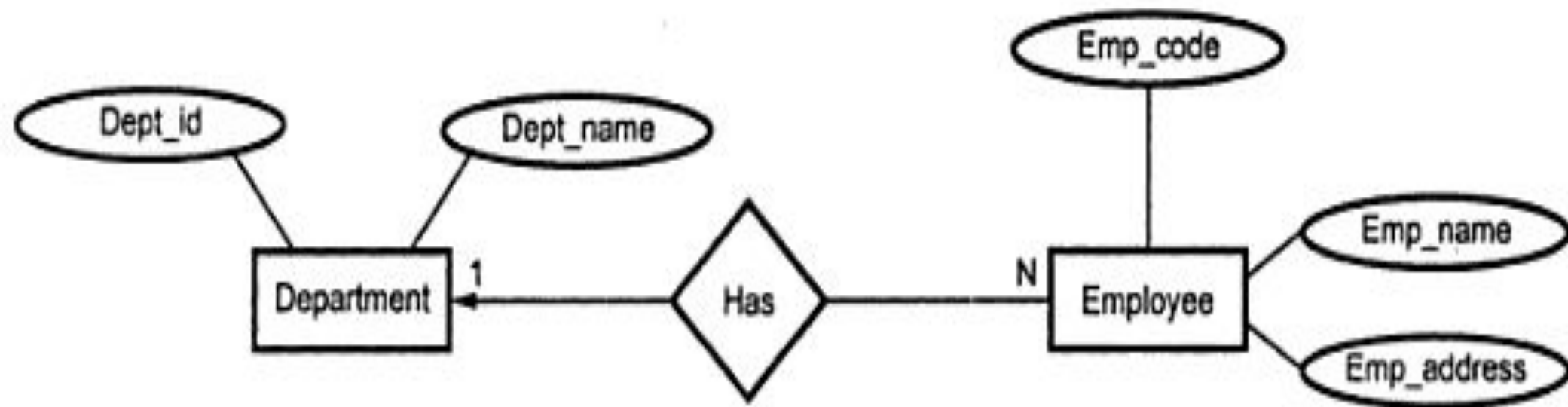
E.g. An instructor may advise many students, but a student may have at most one advisor.

e.g. A student can enrol for many courses

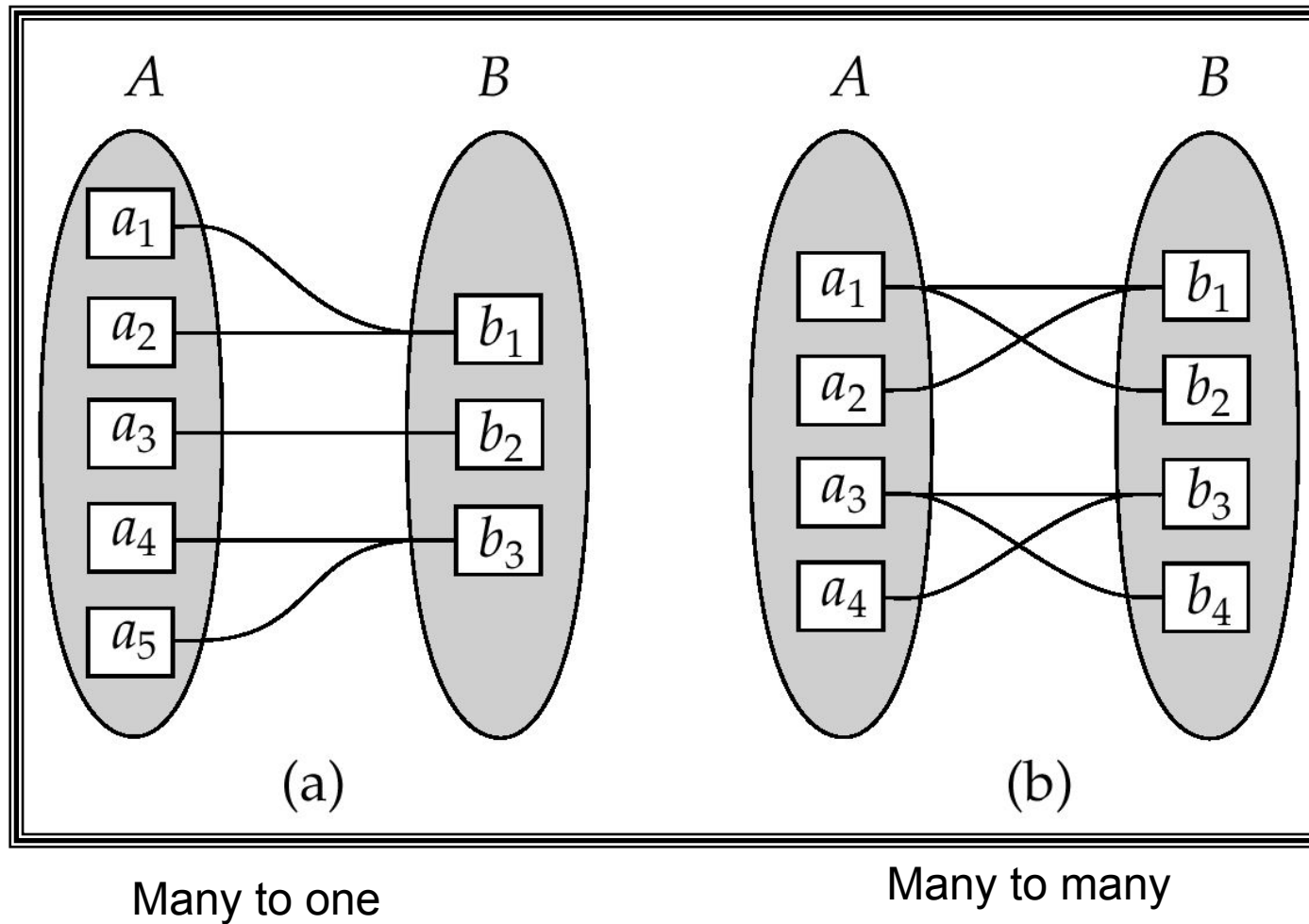
2.One-To-Many E-R model concepts

Constraints

Mapping cardinality Constraints 1 to M



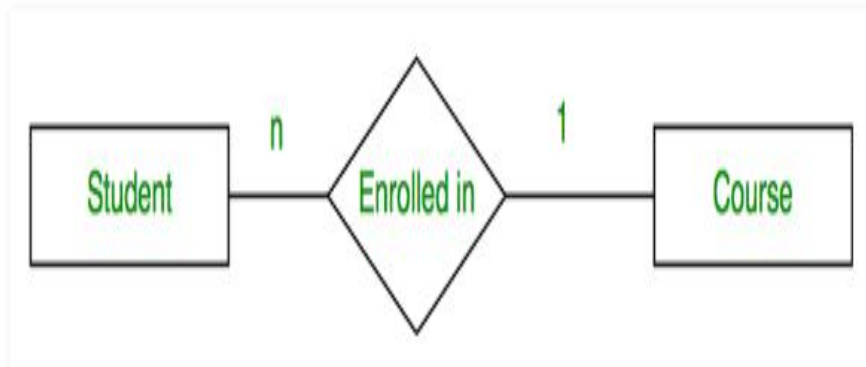
Mapping cardinality Constraints



Mapping Constraints

3. Many-To-One

An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.



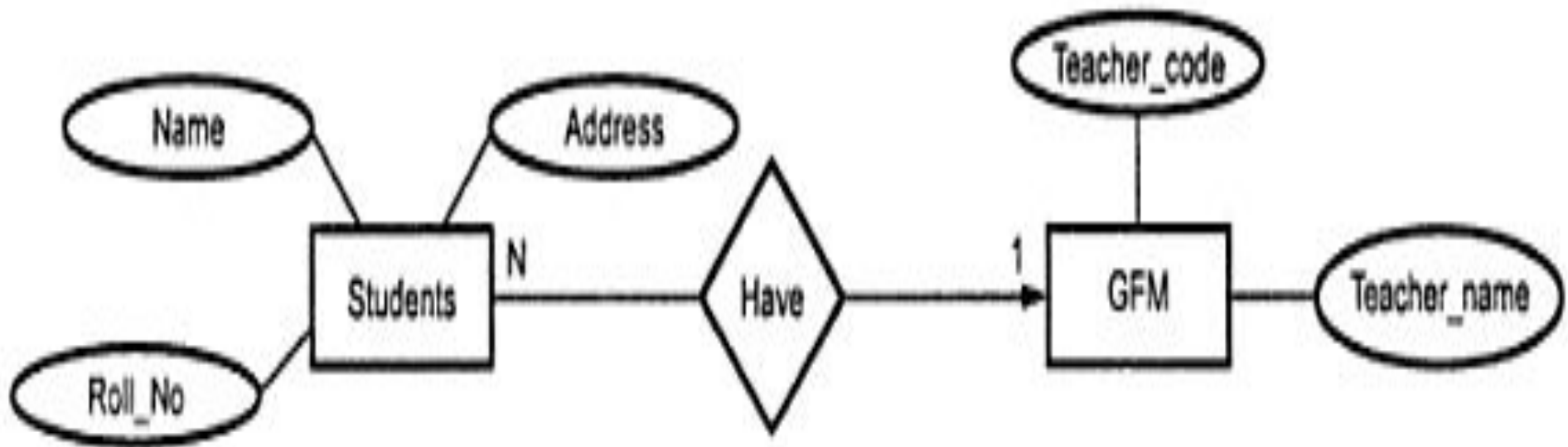
This is another notation which shows student can take at most one course and one course can be taken by many students.

E-R model concepts

3. Many-To-One

Constraints

Mapping cardinality Constraints M to 1

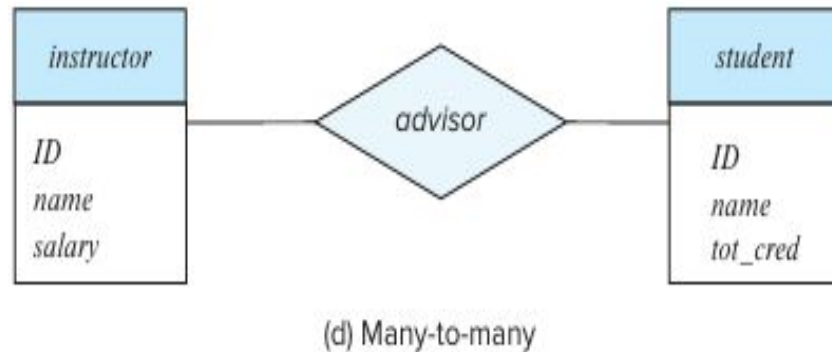


(c) Many-to-one

Mapping Constraints

4.Many-To-Many

An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A.



An instructor may advise many students, and a student may have many advisors.

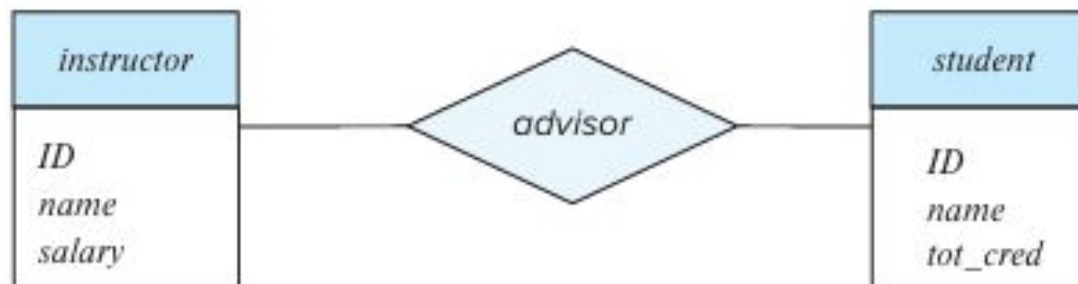
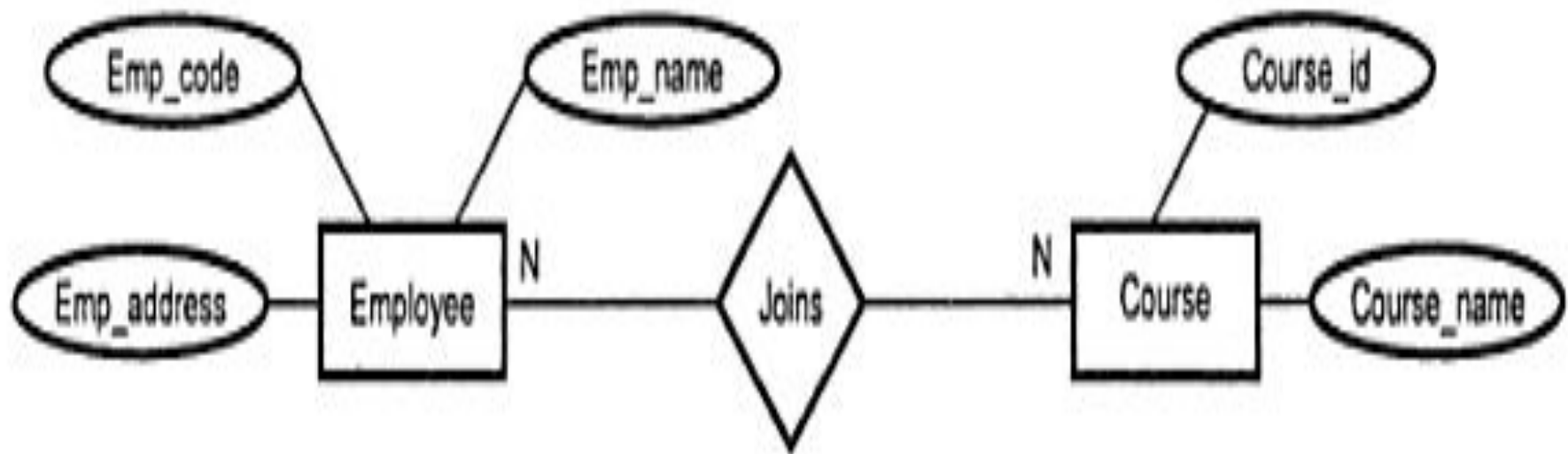
This is another notation to show mapping cardinality. Which shows one student can take many courses and one course can be taken by many students

E-R model concepts

4.Many-To-Many

Constraints

Mapping cardinality Constraints M to M



(d) Many-to-many

Relationship instances of the WORKS_FOR N:1 relationship between EMPLOYEE and DEPARTMENT

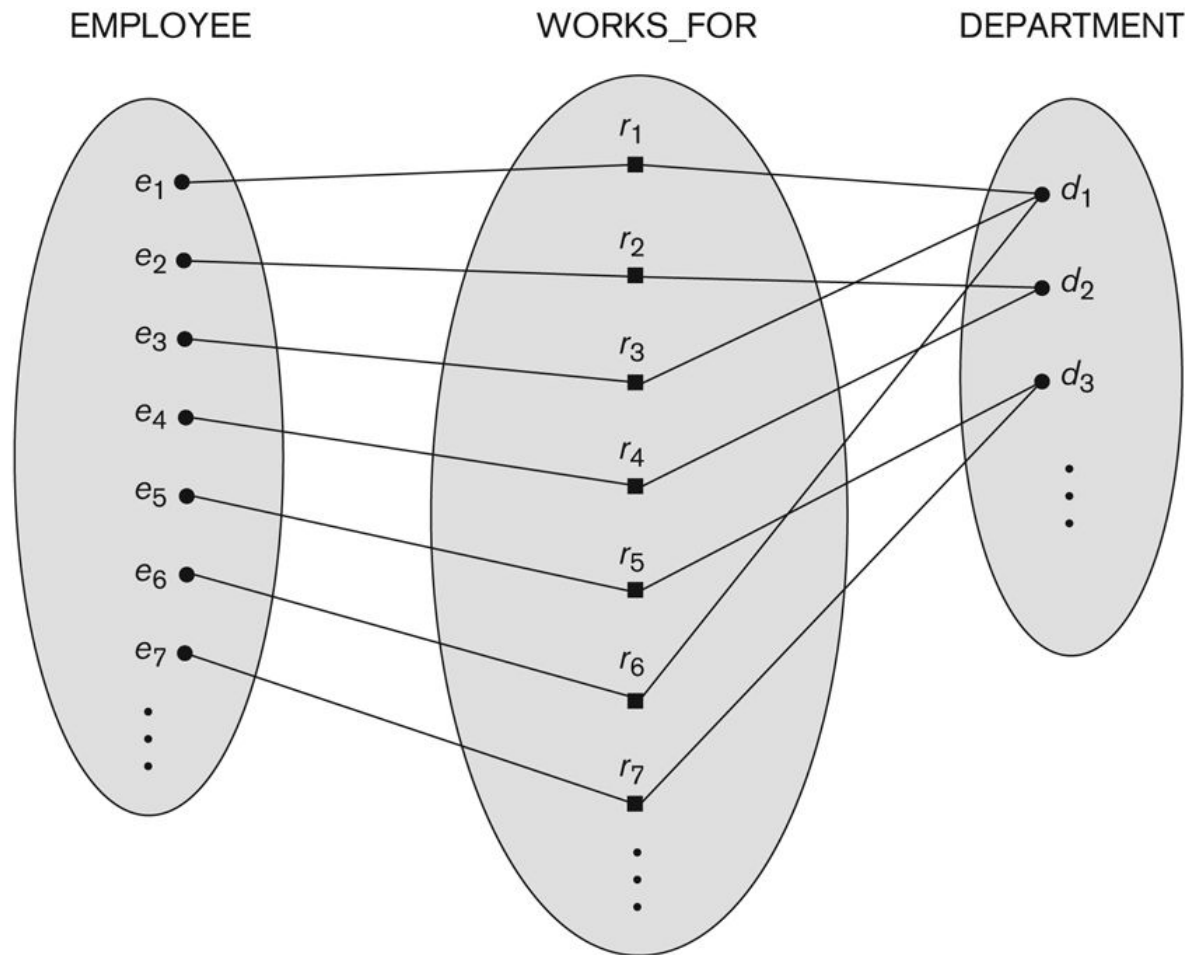


Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Relationship instances of the M:N WORKS_ON relationship between EMPLOYEE and PROJECT

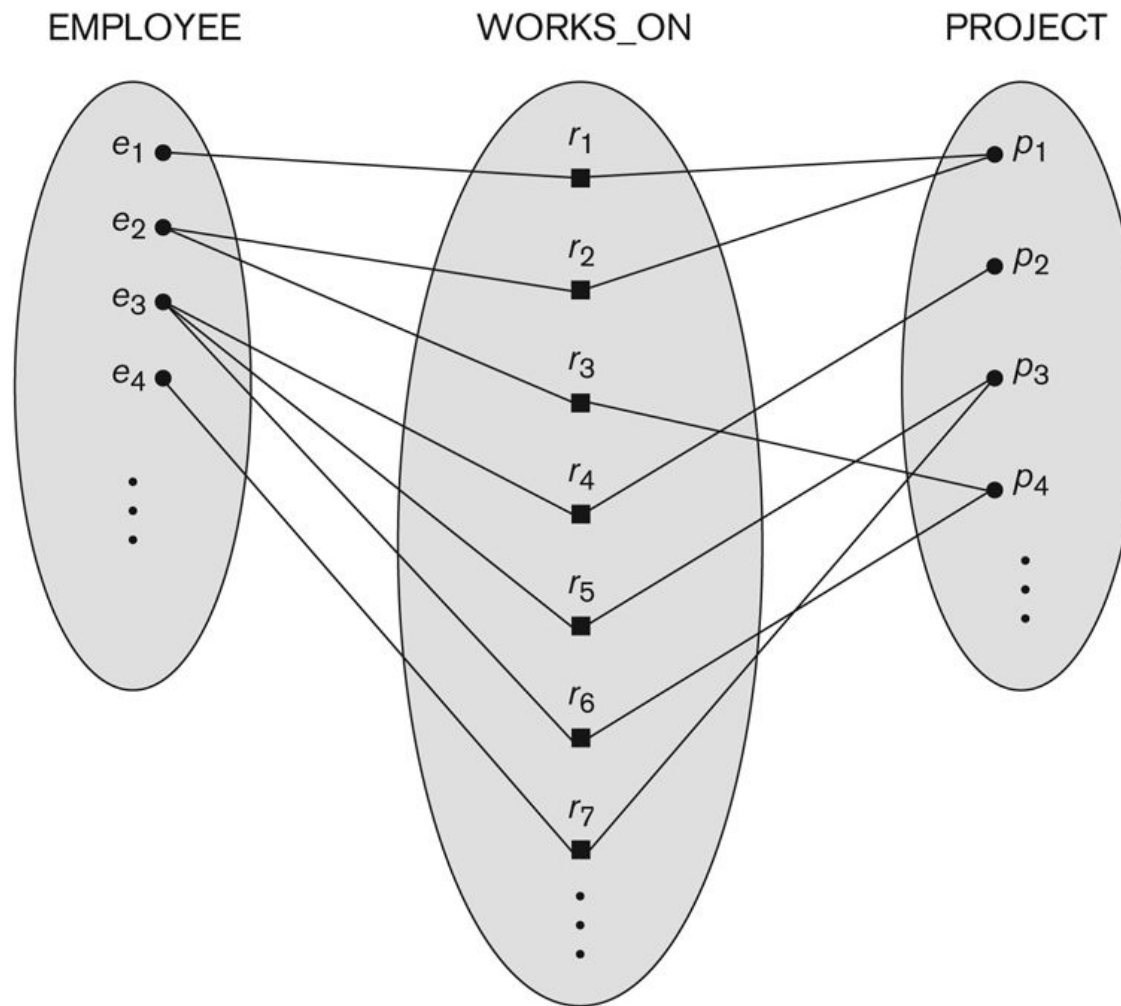


Figure 3.13
An M:N relationship,
WORKS_ON.

Crow foot notation for an ER diagram

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many

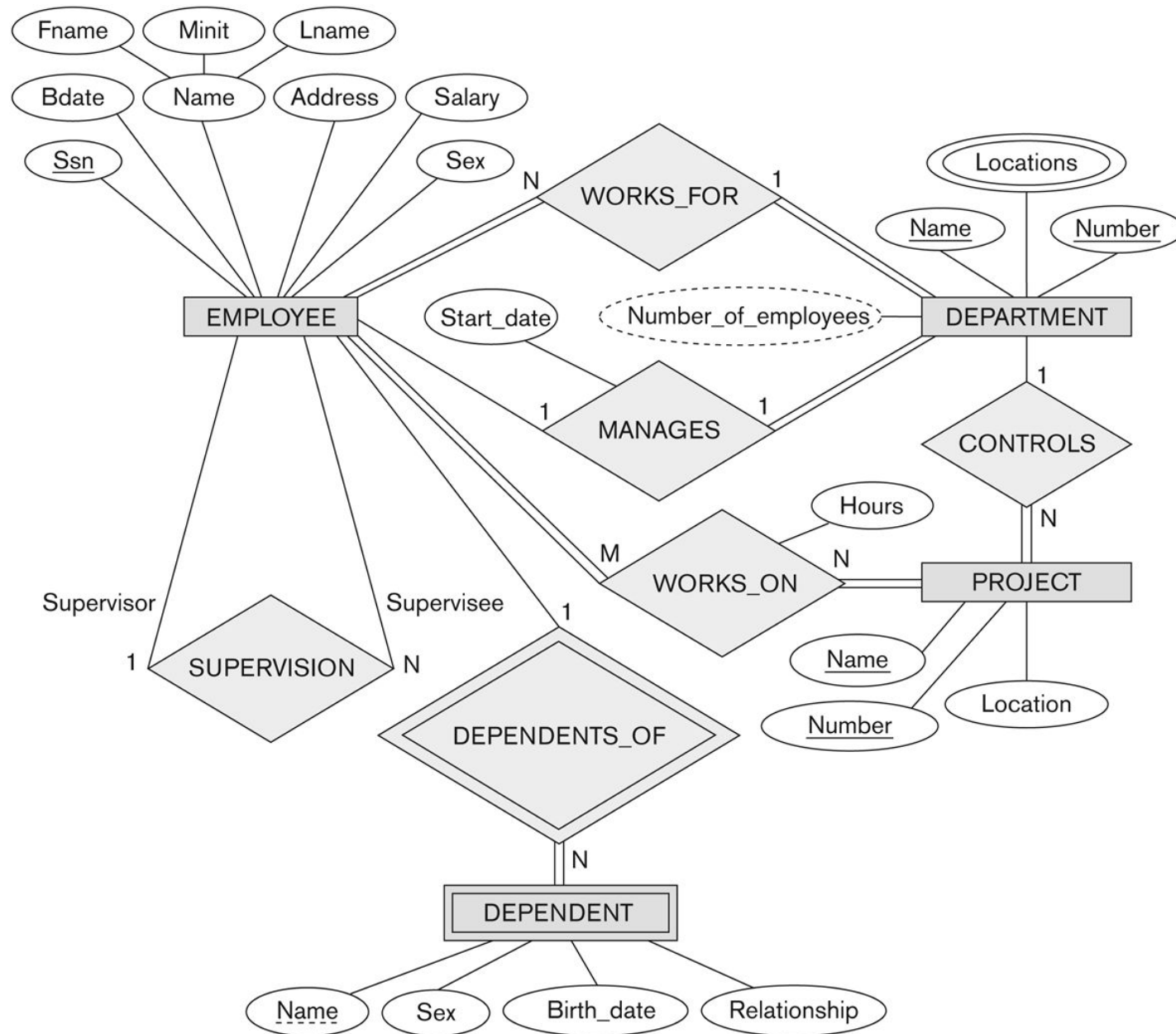
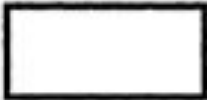
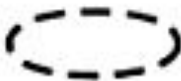
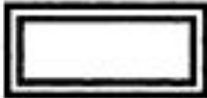


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

E-R model concepts

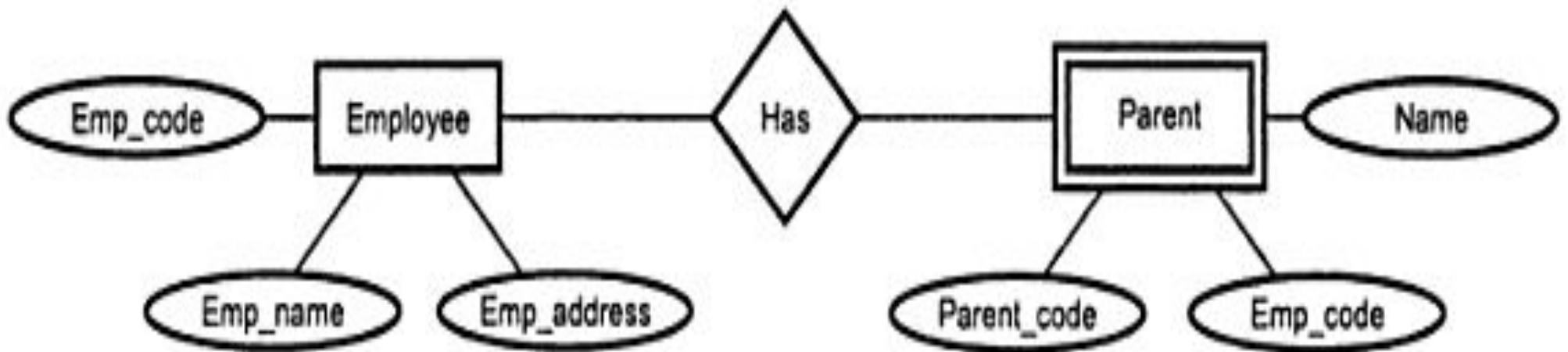
Component name	Symbol	Description *
1. Rectangles		Represents entity sets.
2. Ellipses		Represents attributes.
3. Diamonds		Represents relationship sets.
4. Lines		Links attributes to entity sets & entity sets to relationship sets.
5. Double ellipses		Represents multivalued attributes.
6. Dashed ellipses		Represents derived attributes.
7. Double rectangles		Represents weak entity sets.
8. Double lines		Represents total participation of an entity in a relationship set.

E-R model concepts

Primary Key :

We must have a way to specify how entities within a given **Entity set** and **Relationships** within a given relationship set are distinguished.

Strong and weak entity set



E-R model concepts

- An entity is an object in the miniworld.

For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT

- An attribute of an entity can have a value from a value set (domain)

For example an EMPLOYEE entity may have a Name, SSN, Address, gender, BirthDate

- Each attribute has a *value set* (or data type) associated with it – e.g. integer, string, subrange, enumerated type, ...
- Each entity belongs to some one entity type i.e; entities in one entity type have the same attributes (so each entity type is a set of similar entities).

E-R model concepts (cont..)

- A key attribute of an entity type is one whose value uniquely identifies an entity of that type.
For example, SSN of EMPLOYEE.
- A combination of attributes may form a composite key.
For example, VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- If there is no applicable value for an attribute that attribute is set to a null value.
- An entity type may have more than one key.
For example, the CAR entity type may have two keys:
VehicleIdentificationNumber (popularly called VIN) and
VehicleTagNumber (Number, State), also known as license_plate number.

1. Entity Type / Entity Set

Entity Sets



- ☆ An entity set is a set of entities of the same type that share the same properties, or attributes.
- ☆ Example: The set of all people who are instructors at a given university can be defined as the entity set *Instructor*.
- ☆ It does not need to be disjoint.

ENTITY SET corresponding to the ENTITY TYPE CAR

CAR

Registration(RegistrationNumber, State), VehicleID, Make, Model, Year, (Color)

car_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 1999, (red, black))

car_2
((ABC 123, NEW YORK), WP9872, Nissan 300ZX, 2-door, 2002, (blue))

car_3
((VSY 720, TEXAS), TD729, Buick LeSabre, 4-door, 2003, (white, blue))

▪

▪

▪

Attributes

Attributes

- ☆ Each entity has a **value** for each of its **attributes**.
- ☆ For instance, a particular *Instructor* entity may have:
 - the value **12121** for **ID**,
 - the value **Sujeet** for **Name**,
 - the value **Finance** for **Dept_name**, and
 - the value **90000** for **Salary**.

Attribute Examples

Name = John Doe
Birthdate = May 10, 1989
Age = 42
Degree = null
SSN = 123456789

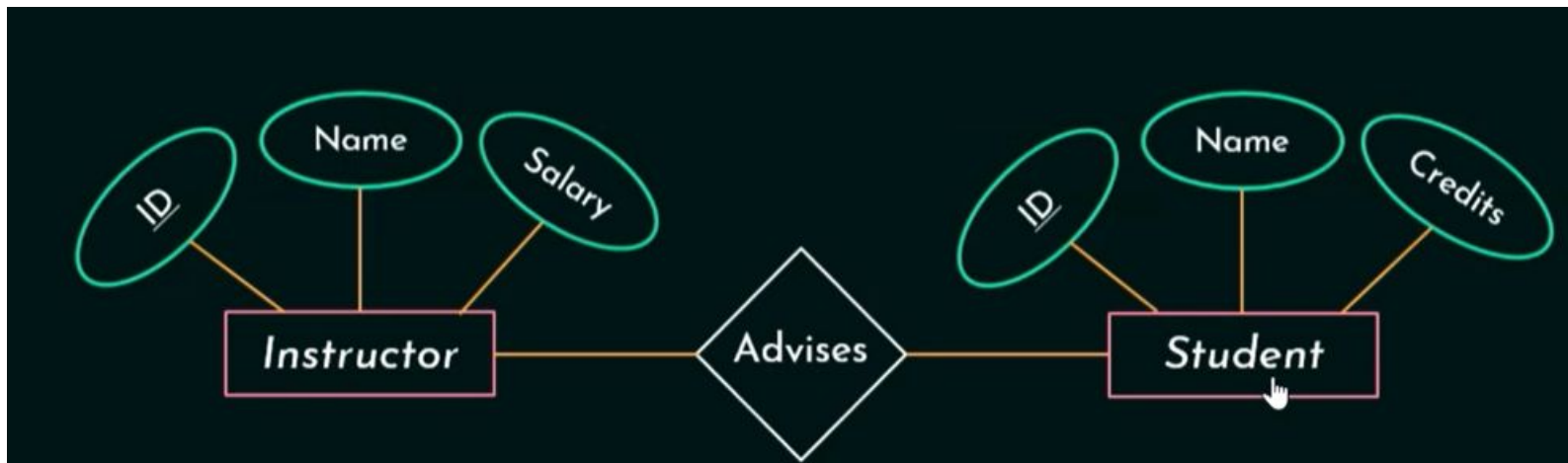
Name = Jane Doe
Birthdate = May 10, 1989
Age = 42
Degree = B.S., M.S.
SSN = 987654321

Name = John Doe
Birthdate = May 10
Birthyear = 1989
Age = 42
Degree = null
SSN = 123456789

2. Relationships and Relationship Types

Relationship Sets

- ☆ It is an **association** among **several entities**.
- ☆ Example: **advises**, **borrows**, **teaches** etc.
- ☆ A **relationship set** is a **set of relationships** of **same type**.
- ☆ $R \subseteq \{ (e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n \}$ is a **relationship**.
- ☆ The **part** an **entity** plays in a **relationship** is the **entity's role**.

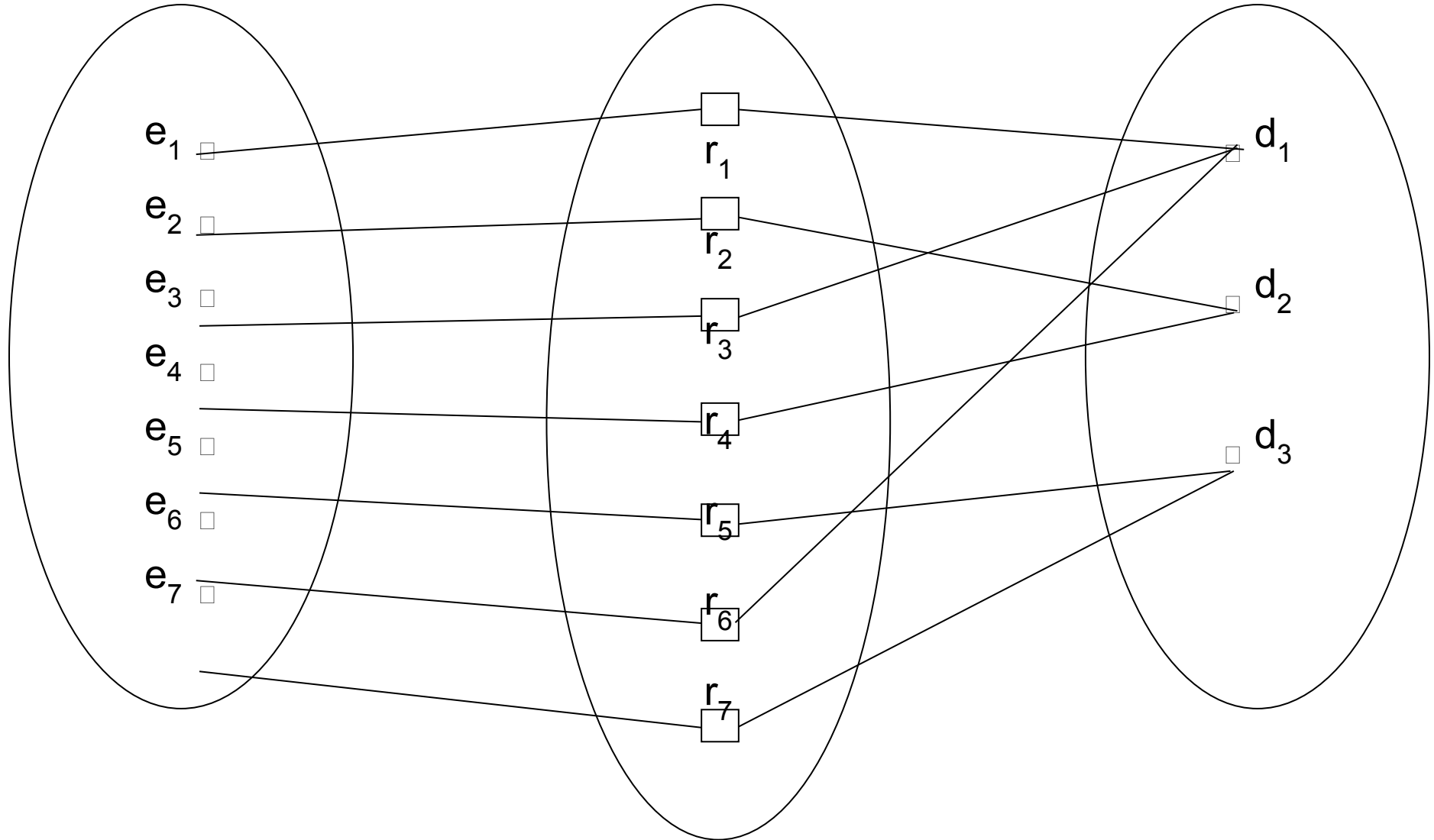


Example relationship instances of the WORKS_FOR relationship between EMPLOYEE and DEPARTMENT

EMPLOYEE

WORKS_FOR

DEPARTMENT



Relationships and Relationship Types

- A relationship relates two or more distinct entities with a specific meaning.
 - For example, EMPLOYEE John Smith works on the ProductX PROJECT, or EMPLOYEE Franklin Wong manages the Research DEPARTMENT.
- Relationships of the same type are grouped or typed into a relationship type.
 - For example, the WORKS_ON relationship type in which EMPLOYEEs and PROJECTs participate, or the MANAGES relationship type in which EMPLOYEEs and DEPARTMENTs participate.
- The degree of a relationship type is the number of participating entity types.
 - Both MANAGES and WORKS_ON are binary relationships.

Relationships and Relationship Types

There are different types of Relationships like:

Unary Relationship

Binary Relationship

Ternary Relationship

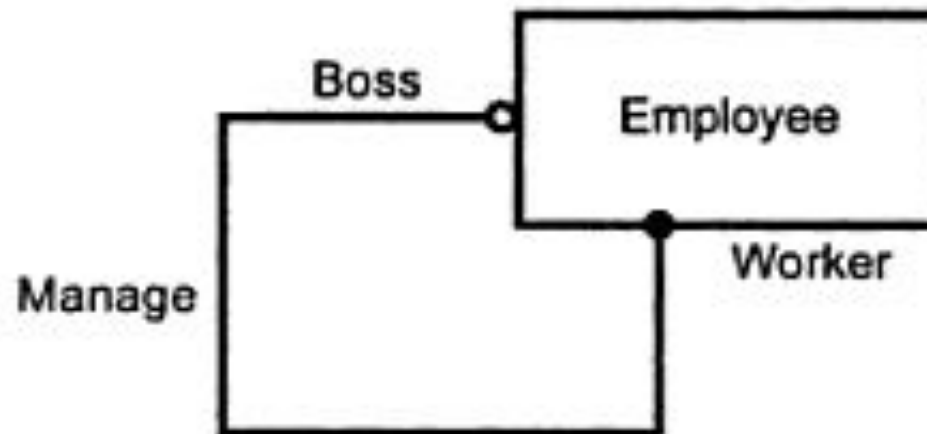
N-Ary Relationship

Recursive Relationship

Relationships and Relationship Types

1. Unary Relationship Set-

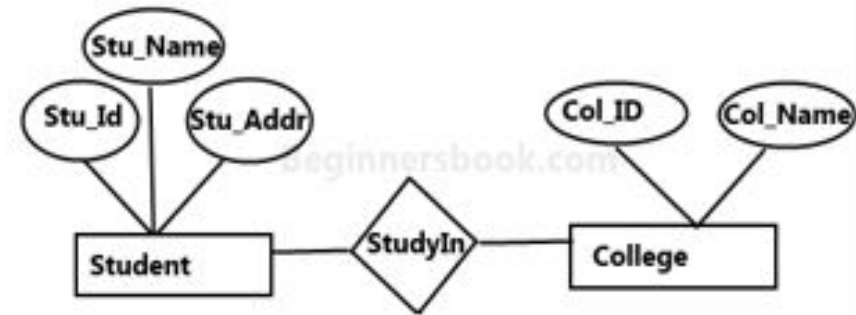
- Unary relationship set is a relationship set where only one entity set participates in a relationship set.



Relationship Types

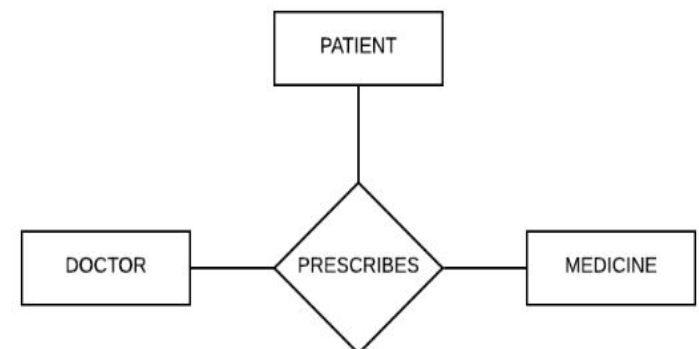
- **Binary Relationship:**

When there is a relationship between two different entities, it is known as a binary relationship.



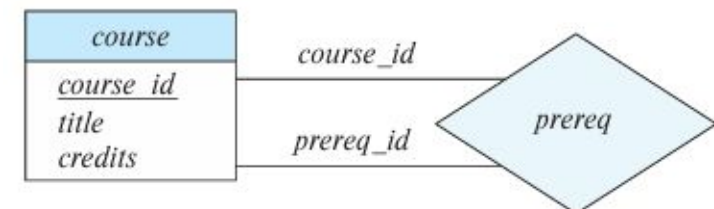
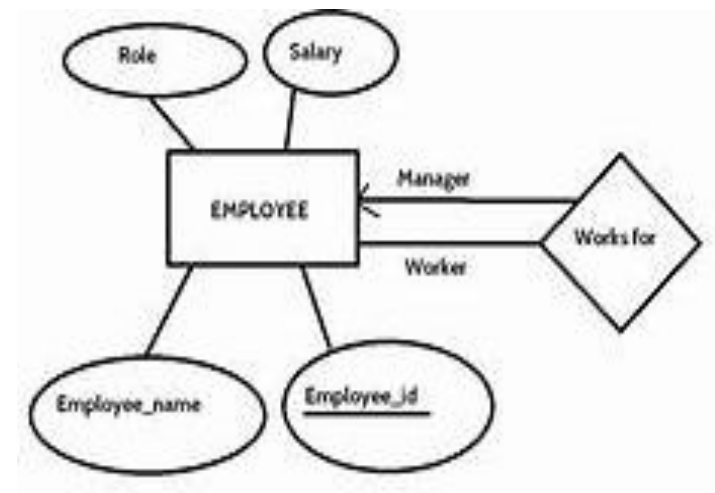
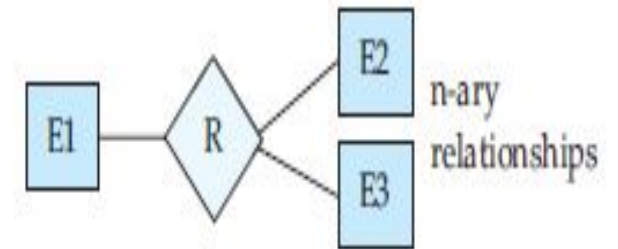
- **Ternary Relationship:**

When there is a relationship between three different entities, it is known as a ternary relationship.



Relationship Types

- **N-Ary Relationship:** When any relationship consists of more than two Entities then it is called N-ary relationship.
- **Recursive Relationship**
 - A relationship between two entities of similar entity type is called a recursive relationship.
 - Here the same entity type participates more than once in a relationship type with a different role for each instance.
- We indicate roles in E-R diagrams by labeling the lines that connect diamonds to rectangles.



3. Weak Entity Types

2. Weak Entity Set-

- A weak entity set is an entity set that does not contain sufficient attributes to uniquely identify its entities.
- In other words, a primary key does not exist for a weak entity set.
- However, it contains a partial key called as a **discriminator**.
- Discriminator can identify a group of entities from the entity set.
- Discriminator is represented by underlining with a **dashed line**.

3. Weak Entity Types

Example: The figure illustrates the use of double lines to indicate that the participation of the (weak) entity set **section** in the relationship **Sec_course** is total, meaning that every section must be related via **sec_course** to some course. Finally, the arrow from **sec_course** to **course** indicates that each section is related to a single course.

In general, a weak entity set must have a **total participation in its identifying relationship set**, and the relationship is many-to-one toward the identifying entity set



Differences between Strong entity set and Weak entity set

Strong entity set	Weak entity set
A single rectangle is used for the representation of a strong entity set.	A double rectangle is used for the representation of a weak entity set.
It contains sufficient attributes to form its primary key.	It does not contain sufficient attributes to form its primary key.
A diamond symbol is used for the representation of the relationship that exists between the two strong entity sets.	A double diamond symbol is used for the representation of the relationship that exists between the strong and weak entity set.
A single line is used for the representation of the connection between the strong entity set and the relationship.	A double line is used for the representation of the connection between the weak entity set and the relationship set.
Total participation may or may not exist in the relationship.	Total participation always exists in the identifying relationship.

Entity-Relationship Design Issues

The notions of an **entity set** and a **Relationship set** are not precise, and it is possible to define a set of entities and the relationships among them in a number of different ways.

E - R design issues

- The relationship among various entities can be represented in many ways
- Basic issues are :
 - Use of entity set vs attributes
 - Use of Entity Sets vs Relationship Sets
 - Binary versus nary Relationship Sets
 - Placement of Relationship Attributes

Use of Entity Set Vs Attributes

- Choice mainly depends on the structure of the enterprise being modeled, and on the semantics associated with the attribute in question

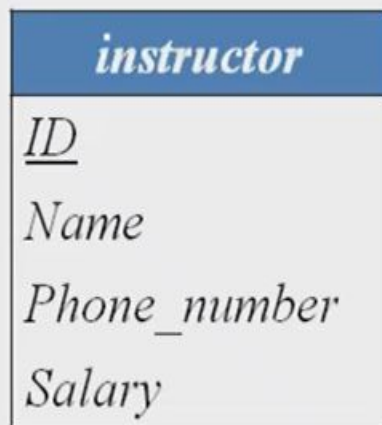


FIG A

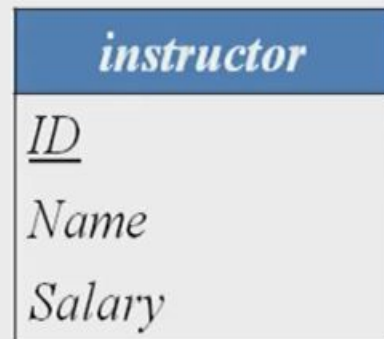
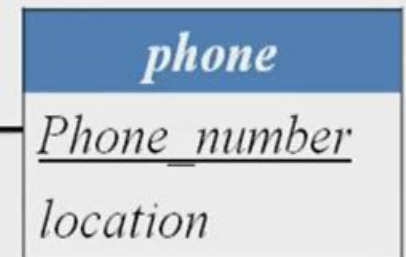


FIG B



Use of Entity Sets Vs Relationship Sets

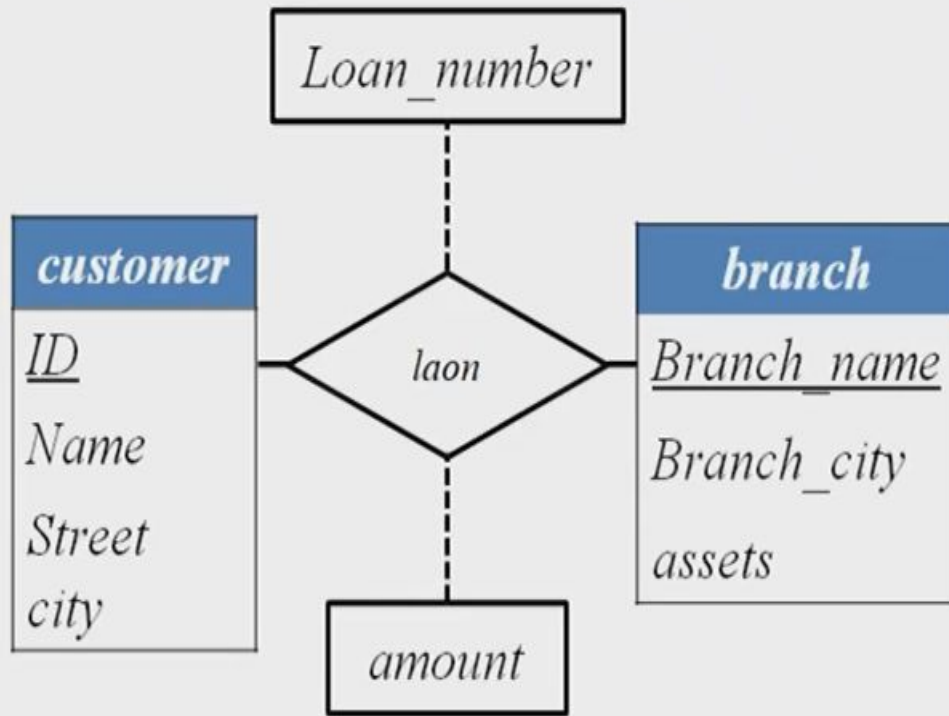


FIG A

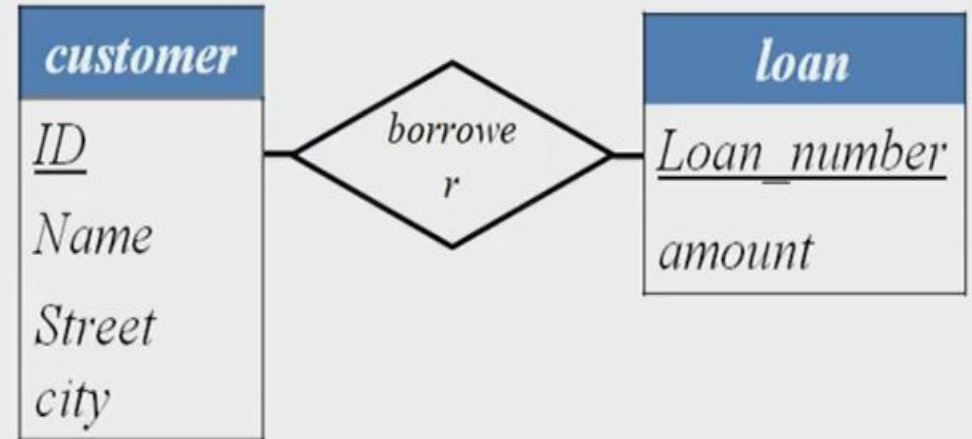


FIG B

Binary Vs *n*-array Relationship Sets

- It is always possible to replace a non-binary relationship set by a number of distinct binary relationship set

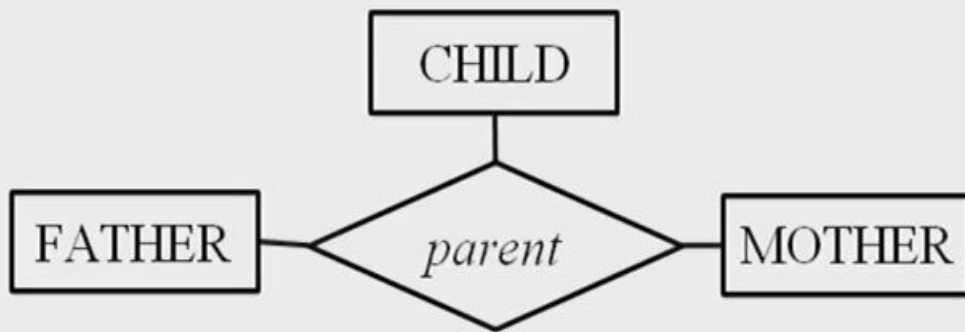


FIG A

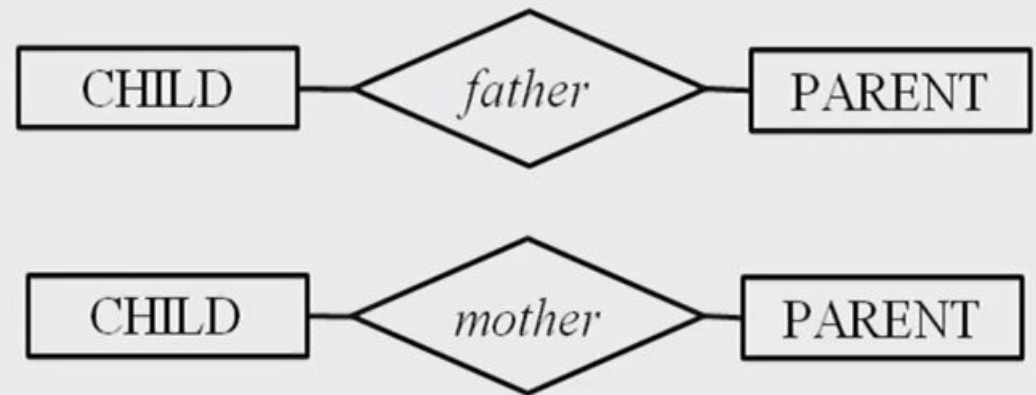
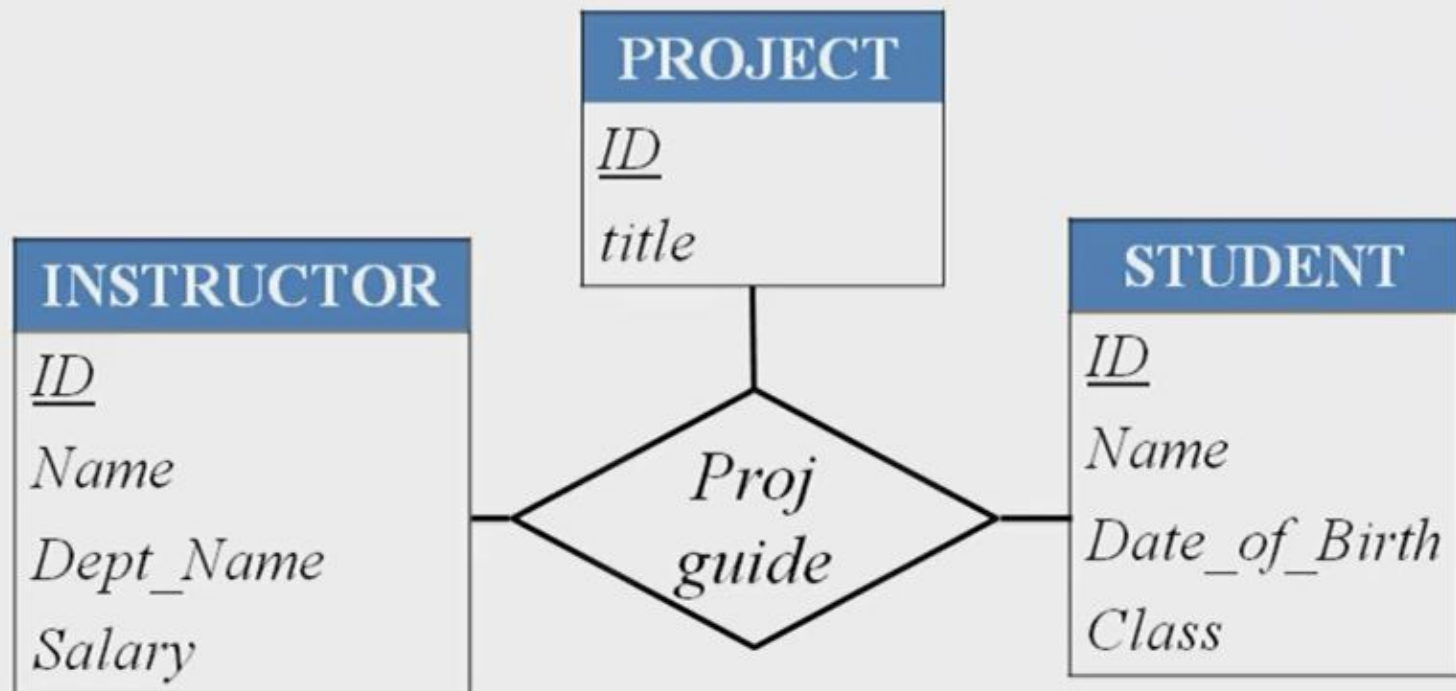


FIG B

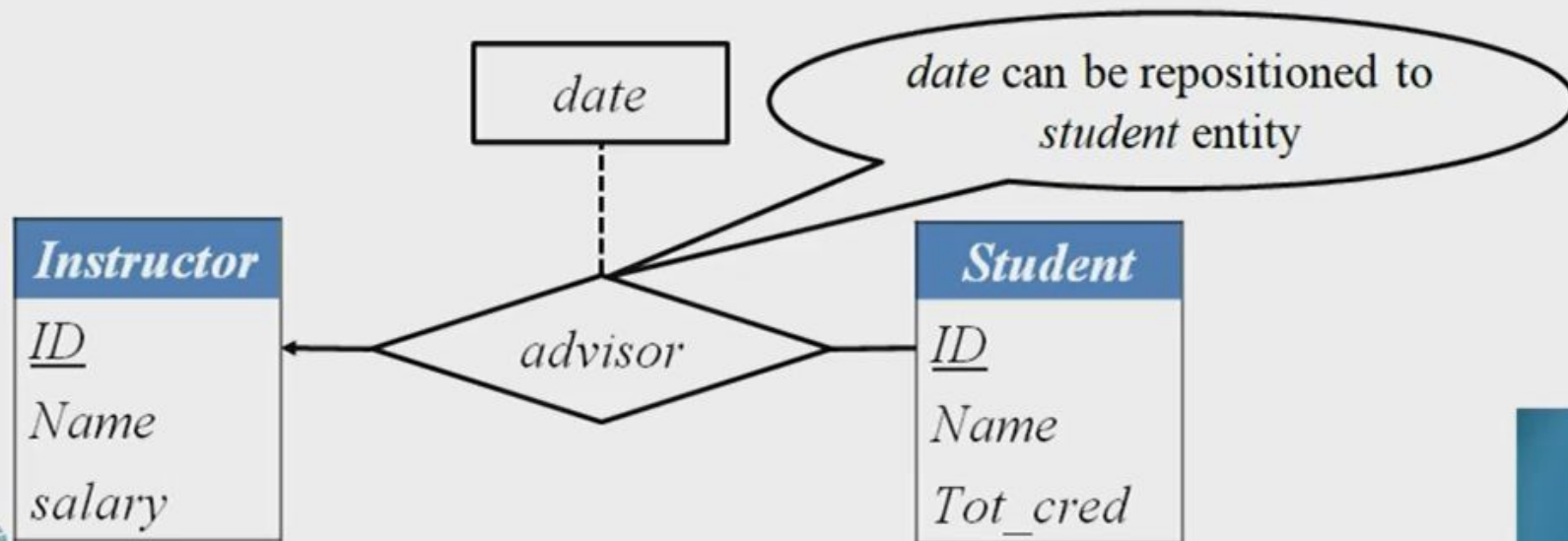
Binary Vs *n*-array Relationship Sets

- There are some relationship that are naturally non-binary
- Eg: *proj_guide*



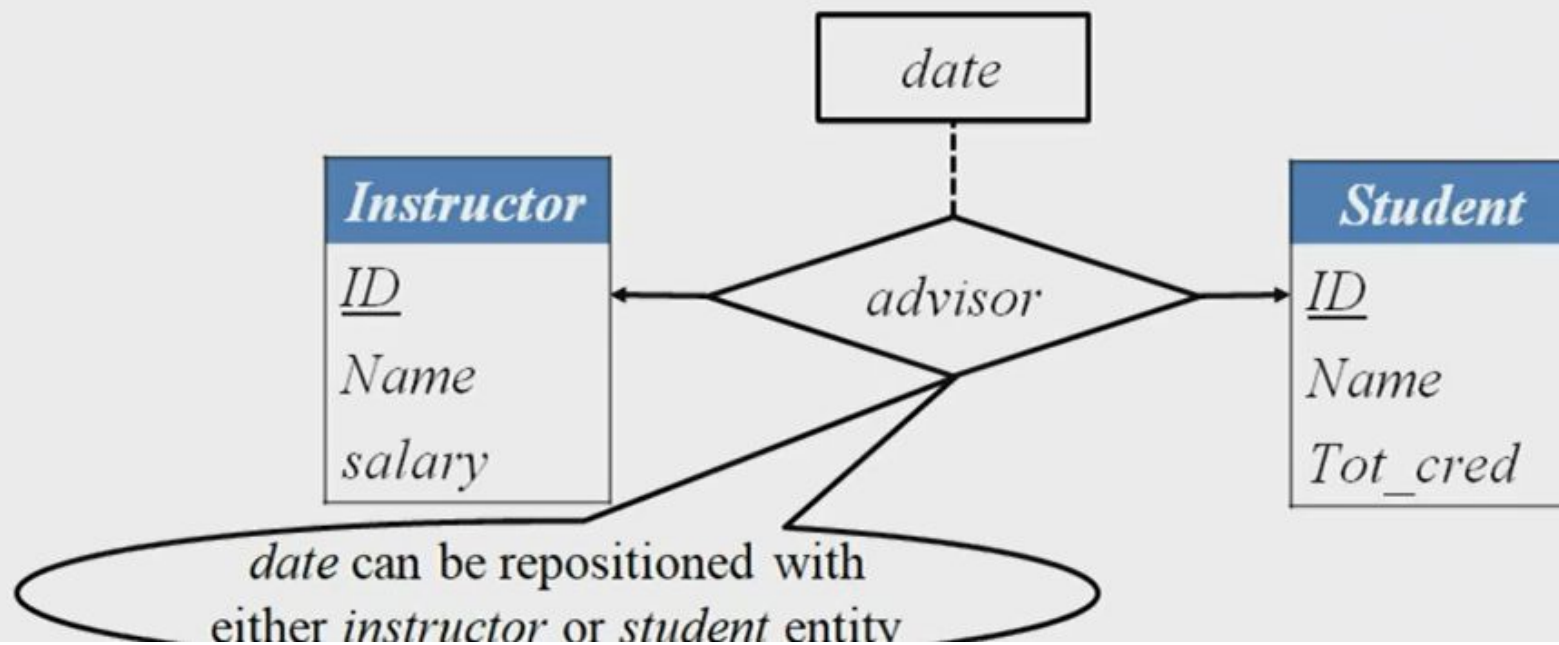
Placement of Relationship Attributes

- Attributes of one – to – many relationship sets can be repositioned to only the entity set on the many side of the relationship.



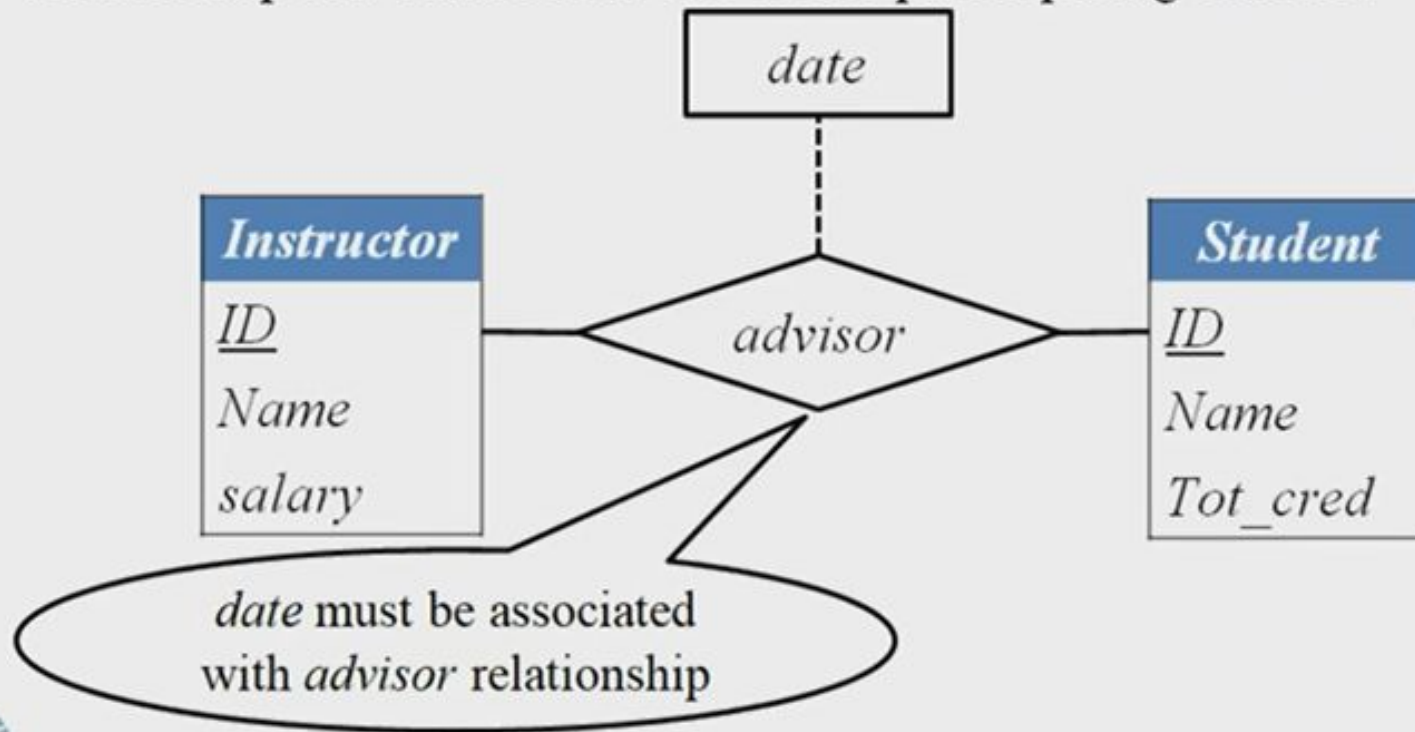
Placement of Relationship Attributes

- For one – to – one relationship sets, the relationship attribute can be associated with either one of the participating entities.



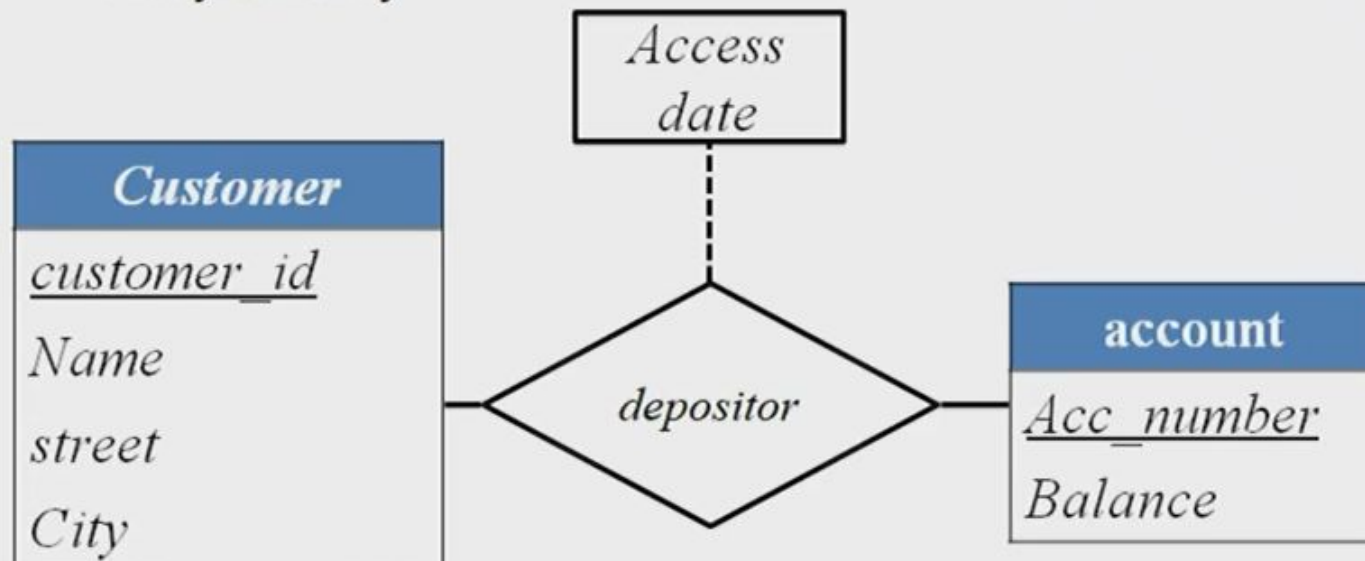
Placement of Relationship Attributes

- For many – to – many relationship set, attributes must be associated with relationship sets rather than one of the participating entities.



THINK AND WRITE

- For the following ER Diagram specify the attribute placement of relationship
 - One-to-one
 - One-to-many
 - Many-to-many



THINK AND WRITE

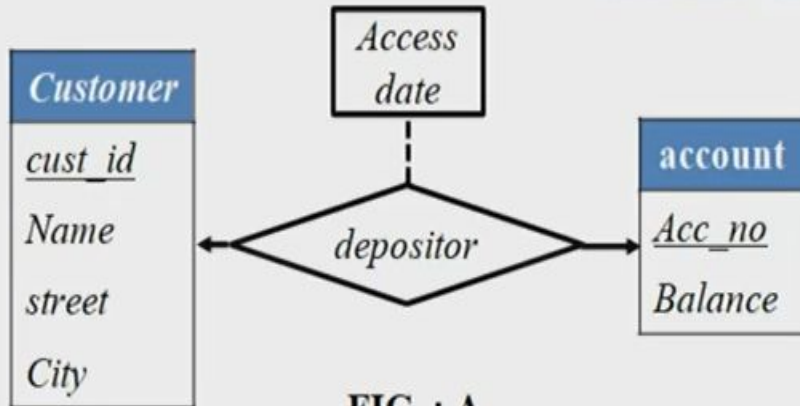


FIG. : A

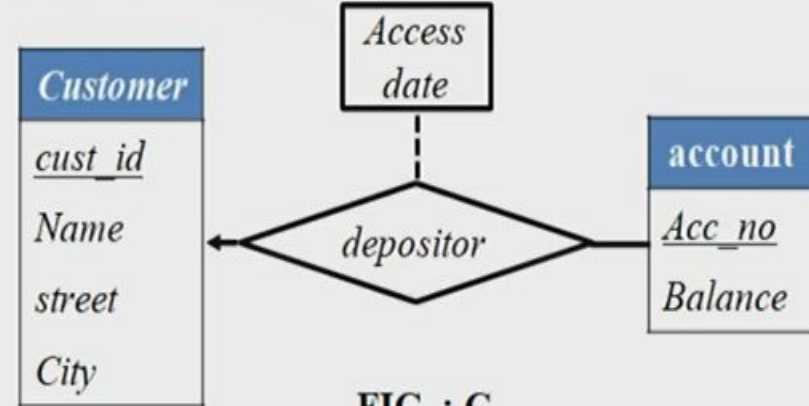


FIG. : C

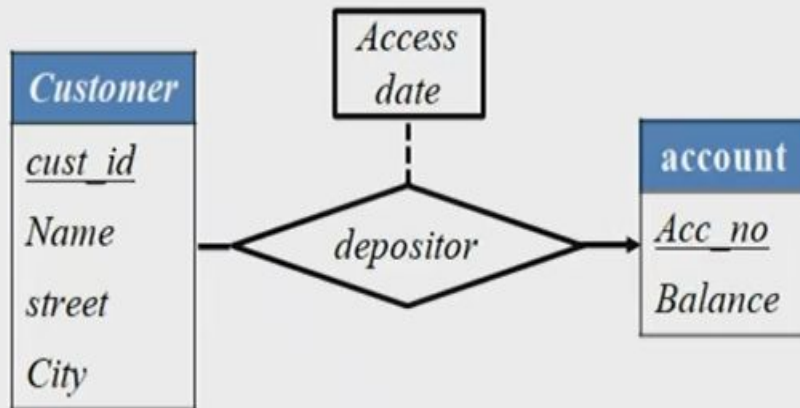


FIG. : B

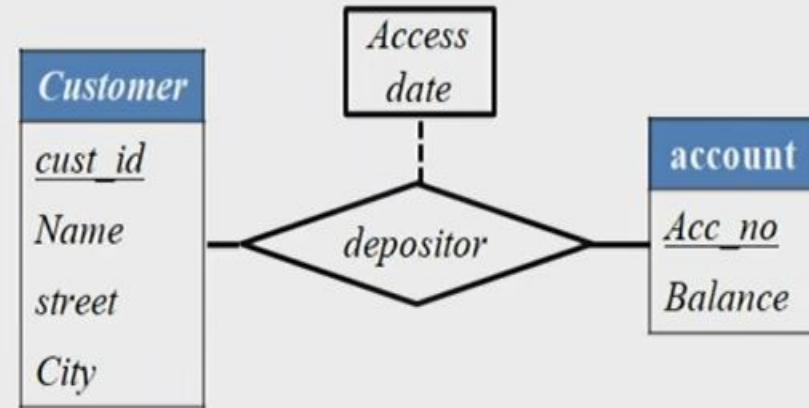


FIG. : D

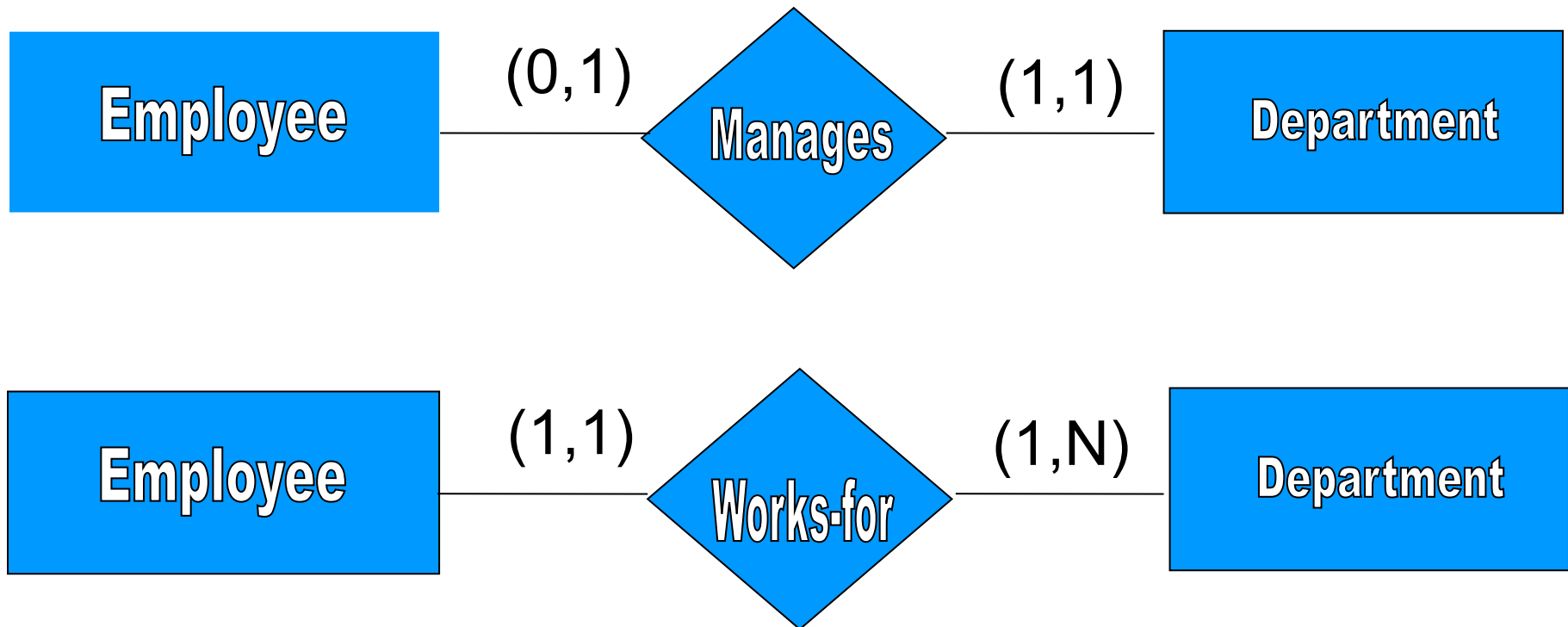
Alternative (min, max) notation for relationship structural constraints:

- Specified on *each participation* of an entity type E in a relationship type R
- Specifies that each entity e in E participates in *at least* min and *at most* max relationship instances in R
- Default(no constraint): min=0, max=n
- Must have $\text{min} \leq \text{max}$, $\text{min} \geq 0$, $\text{max} \geq 1$
- Derived from the knowledge of mini-world constraints

Examples:

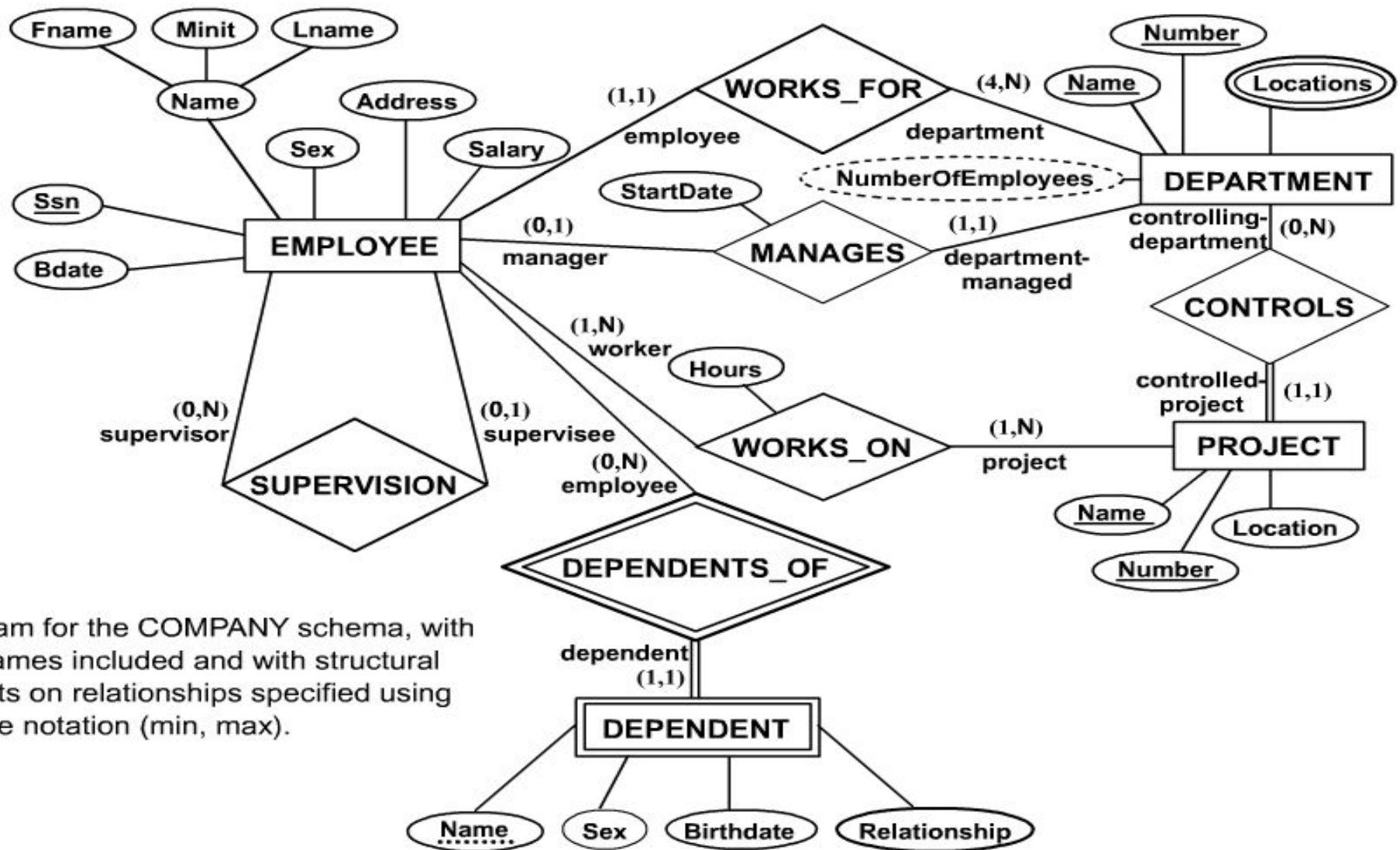
- A department has *exactly one* manager and an employee can manage *at most one* department.
 - Specify (0,1) for participation of EMPLOYEE in MANAGES
 - Specify (1,1) for participation of DEPARTMENT in MANAGES
- An employee can work for *exactly one* department but a department can have *any number of employees*.
 - Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
 - Specify (1,n) for participation of DEPARTMENT in WORKS_FOR

The (min,max) notation relationship constraints



COMPANY ER Schema Diagram using (min, max) notation

Alternative ER Notations



ER diagram for the COMPANY schema, with all role names included and with structural constraints on relationships specified using alternative notation (min, max).

Problems with Current Modeling Tools

DIAGRAMMING

- Poor conceptual meaningful notation.
- To avoid the problem of layout algorithms and aesthetics of diagrams, they prefer boxes and lines and do nothing more than represent (primary-foreign key) relationships among resulting tables.(a few exceptions)

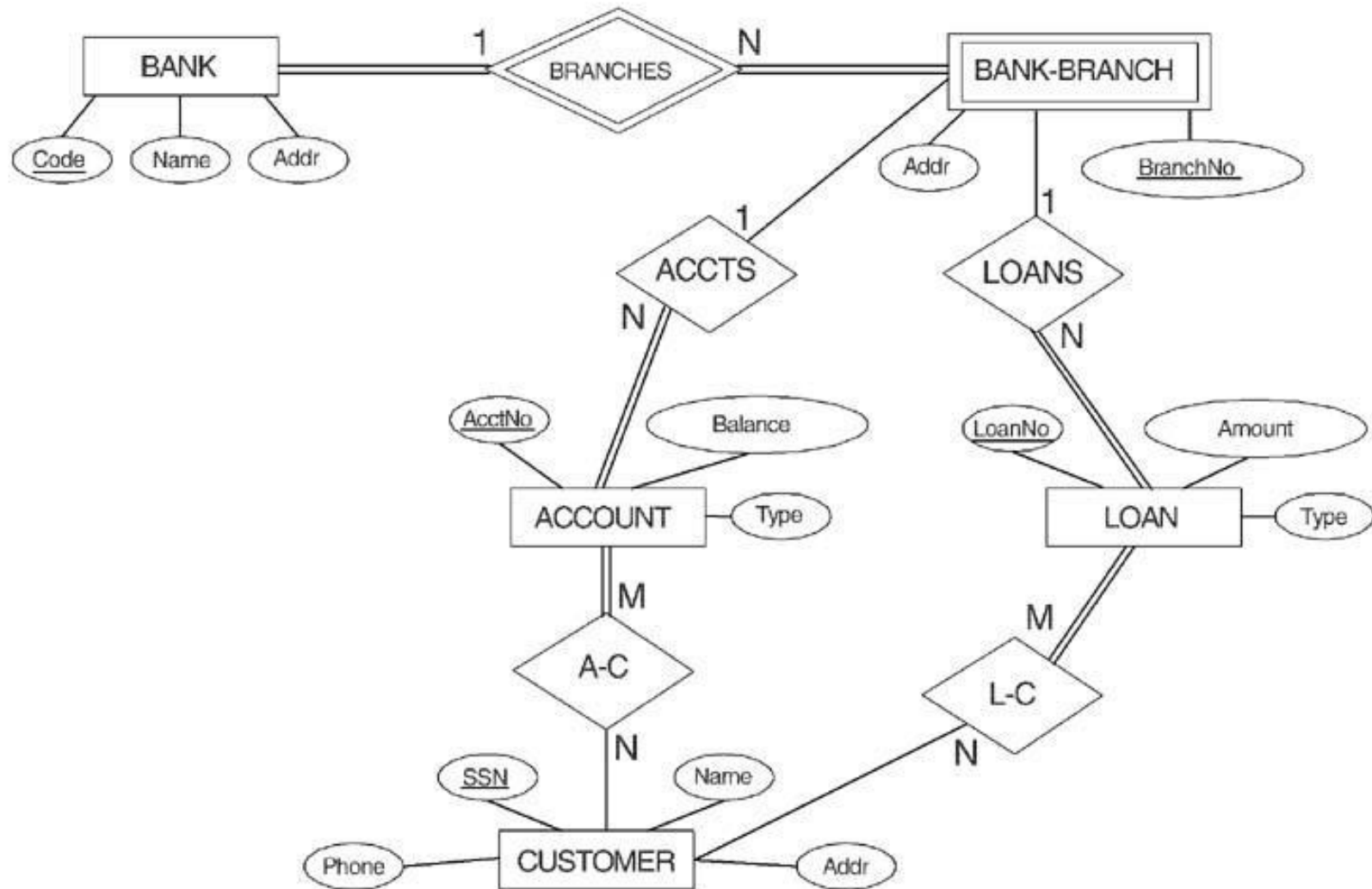
METHODOLOGY

- ☐ lack of built-in methodology support.
- ☐ poor tradeoff analysis or user-driven design preferences.
- ☐ poor design verification and suggestions for improvement.

Some of the Currently Available Automated Database Design Tools

COMPANY	TOOL	FUNCTIONALITY
Oracle	Developer 2000 and Designer 2000	Database modeling, application development
Popkin Software	System Architect 2001	Data modeling, object modeling, process modeling, structured analysis/design
Persistence Inc.	Pwertier	Mapping from O-O to relational model
Rational	Rational Rose	Modeling in UML and application generation in C++ and JAVA
Rogue Ware	RW Metro	Mapping from O-O to relational model
Resolution Ltd.	Xcase	Conceptual modeling up to code maintenance
Sybase	Enterprise Application Suite	Data modeling, business logic modelling
Visio	Visio Enterprise	Data modeling, design and reengineering Visual Basic and Visual C++

ER DIAGRAM FOR A BANK DATABASE



PROBLEM with ER notation



**THE ENTITY RELATIONSHIP MODEL IN
ITS ORIGINAL FORM DID NOT
SUPPORT THE SPECIALIZATION/
GENERALIZATION ABSTRACTIONS**

Extended Entity-Relationship (EER) Features

Incorporates Set-subset relationships

Incorporates Specialization/Generalization Hierarchies

NEXT CHAPTER ILLUSTRATES HOW THE ER MODEL
CAN BE EXTENDED WITH

- Set-subset relationships and
Specialization/Generalization Hierarchies and how to
display them in EER diagrams

Extended Entity Relationship (ER) Features

- As the complexity of data increased in the late 1980s, it became more and more difficult to use the traditional ER Model for database modelling. Hence some improvements or enhancements were made to the existing ER Model to make it able to handle the complex applications better.
- Hence, as part of the **Extended ER Model**, along with other improvements, three new concepts were added to the existing ER Model:
 1. **Generalization**
 2. **Specialization**
 3. **Aggregation**

Generalization



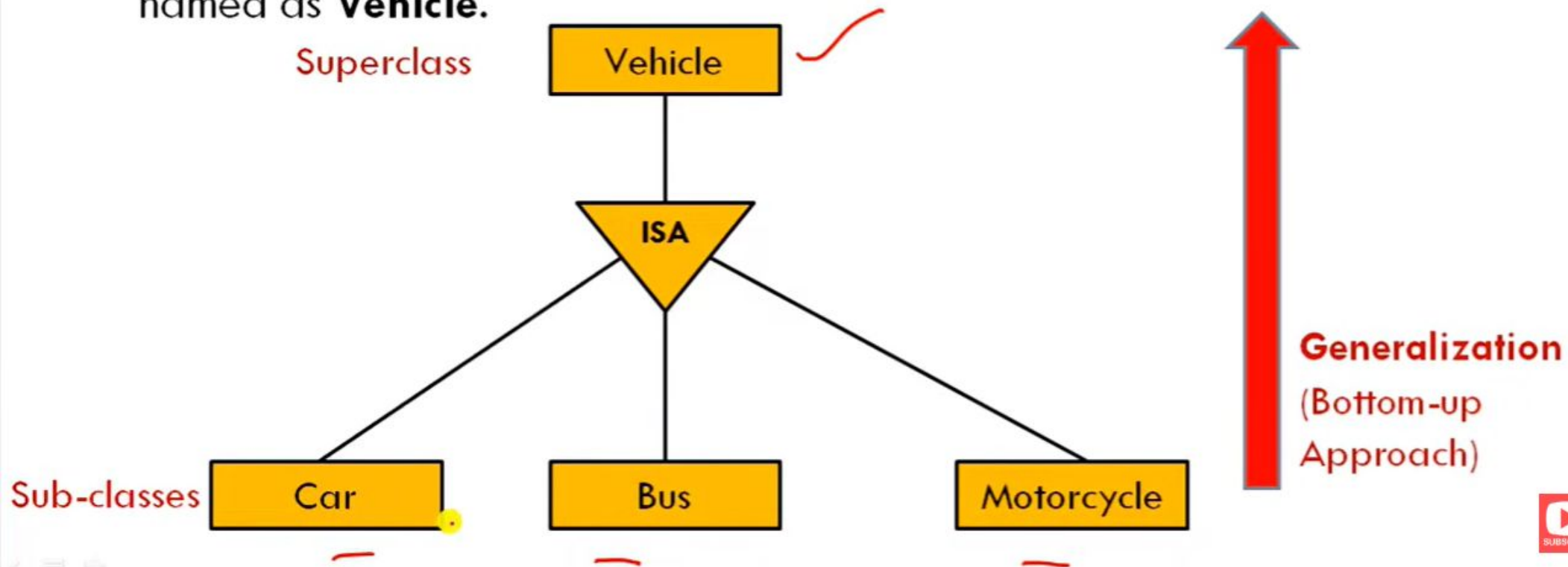
- **Generalization** is the process of extracting common properties from a set of entities and create a generalized entity from it.
- **Generalization is a “bottom-up approach”** in which two or more entities can be combined to form a higher level entity if they have some attributes in common.
 - subclasses are combined to make a superclass.
- **Generalization is used** to emphasize the similarities among lower-level entity set and to hide differences in the schema



Example: Generalization



- Consider we have 3 sub entities Car, Bus and Motorcycle. Now these three entities can be generalized into one higher-level entity (or super class) named as **Vehicle**.



Specialization



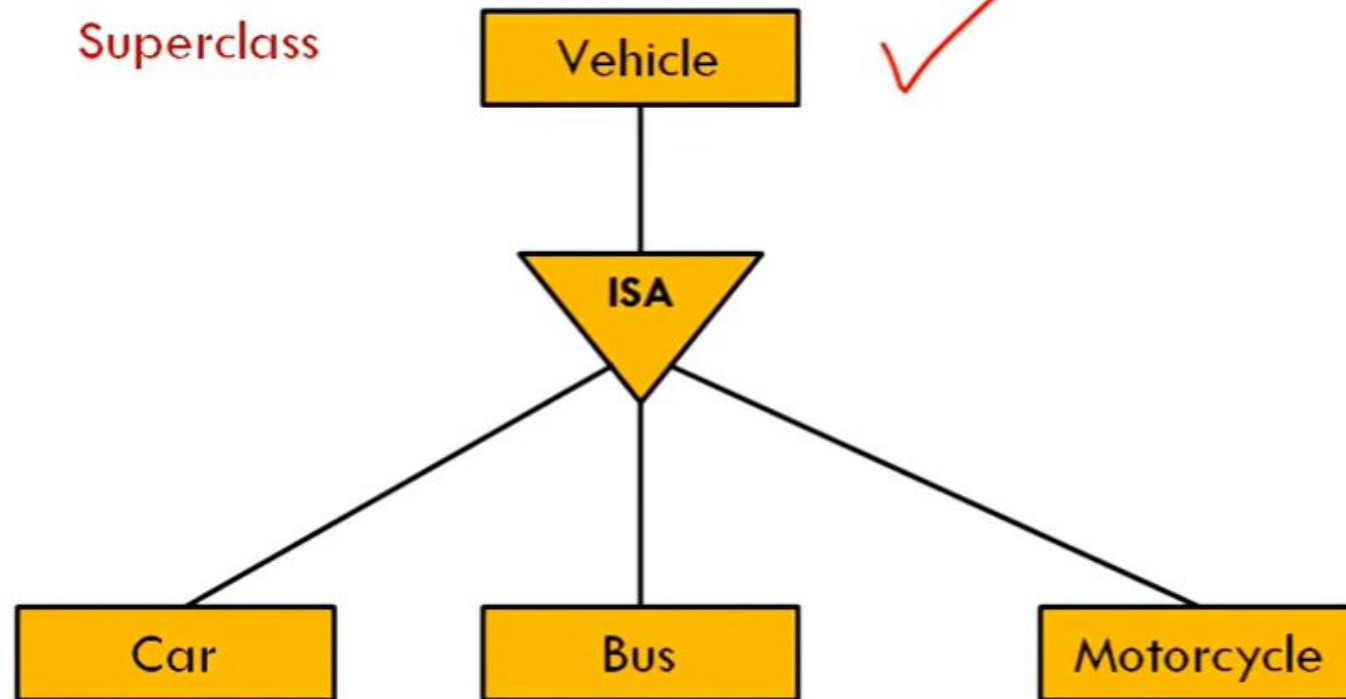
- Specialization is **opposite** of Generalization
- In **Specialization**, an entity is broken down into sub-entities based on their characteristics.
- **Specialization is a “Top-down approach”** where higher level entity is specialized into two or more lower level entities.
- **Specialization is used** to identify the subset of an entity set that shares some distinguishing characteristics.
- **Specialization** can be repeatedly applied to refine the design of schema
- depicted by triangle component labeled **ISA**

Example: Specialization



- **Vehicle** entity can be a Car, Truck or Motorcycle.
 - Normally, the superclass is defined first, the subclass and its related attributes are defined next, and relationship set are then added.

Superclass



Specialization
(Top-down
Approach)



Sub-classes



Inheritance

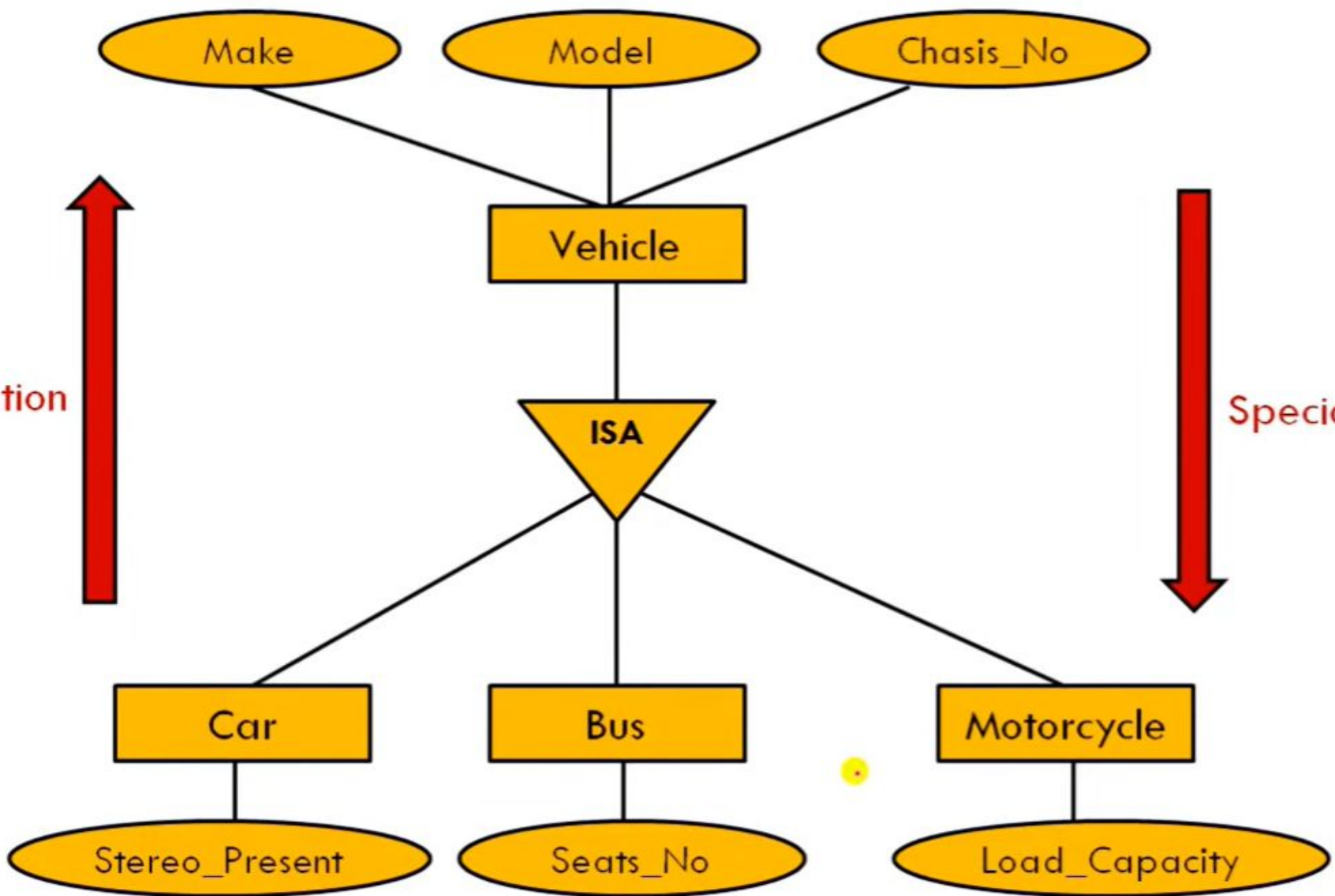


- **Inheritance** is an important feature of generalization and specialization.
- **Attribute inheritance** allows lower level entities to **inherit the attributes** of higher level entities.
 - **For example**, Consider relations Car and Bus inheriting the attributes of Vehicle. Thus, Car is described by attributes of super-class Vehicle as well as its own attributes.
- This also extends to **Participation Inheritance** in which **relationships involving higher-level entity-sets are also inherited by lower-level entity-sets.**
 - A lower-level entity-set can participate in its own relationship-sets, too

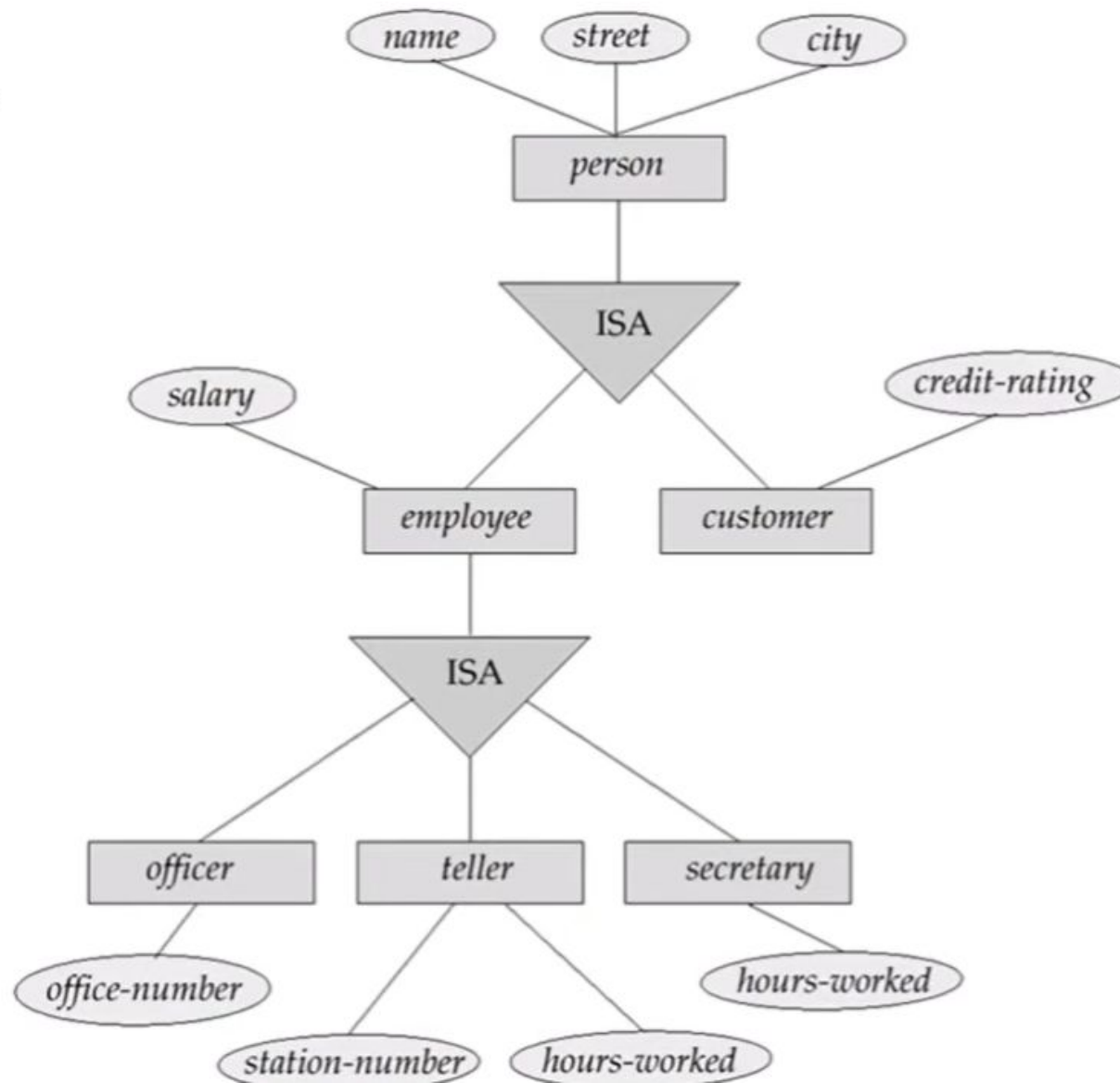


Generalization

Specialization



Example



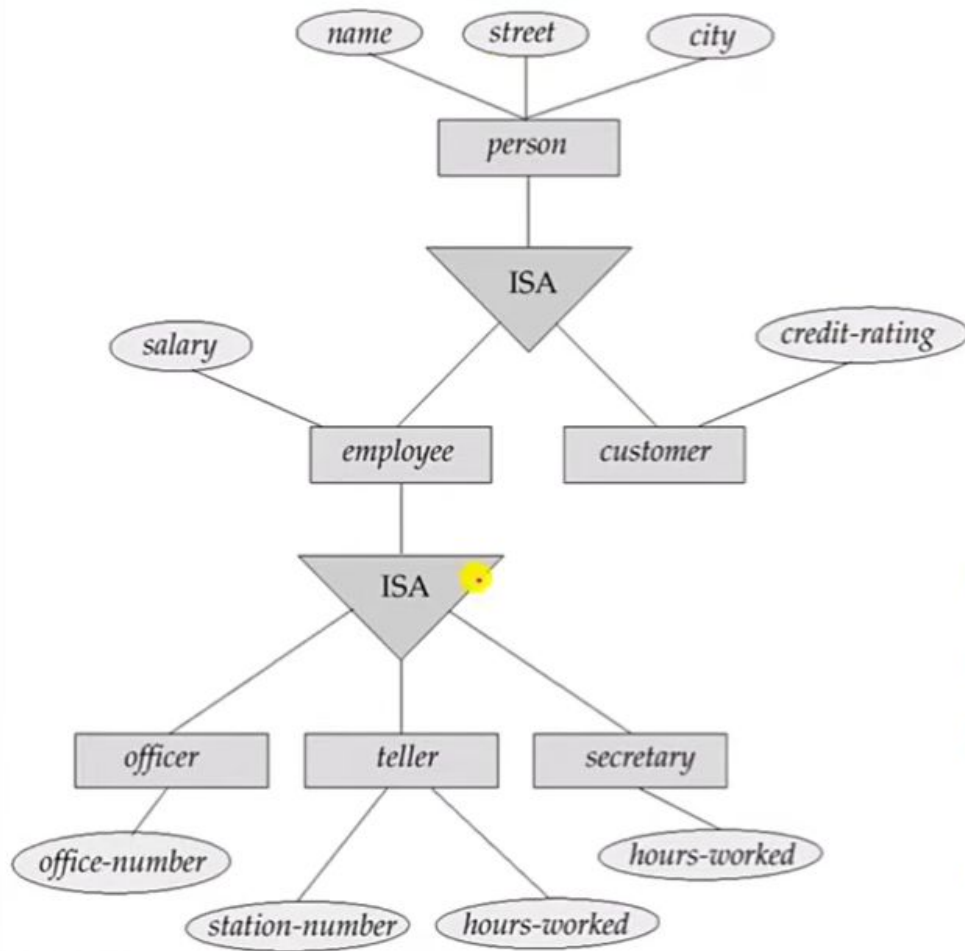
Generalization

Specialization



Example

How Schema or Tables can be formed?



Four tables can be formed:

1. **customer** (name, street, city, credit_rating)
2. **officer** (name, street, city, salary, office_number)
3. **teller** (name, street, city, salary, station_number, hours_worked)
4. **secretary** (name, street, city, salary, hours_worked)



Aggregation

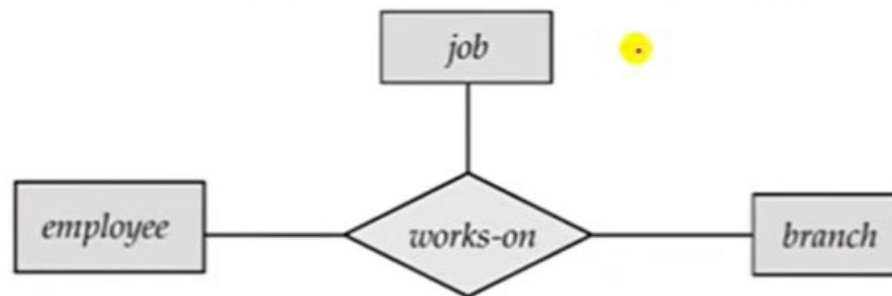


- **Aggregation** is used when we need to express a **relationship among relationships**
- **Aggregation** is an **abstraction** through which **relationships are treated as higher level entities**
- **Aggregation** is a process when a **relationship between two entities is considered as a single entity** and again this single entity has a **relationship with another entity**

Example: (Relationship of Relations)



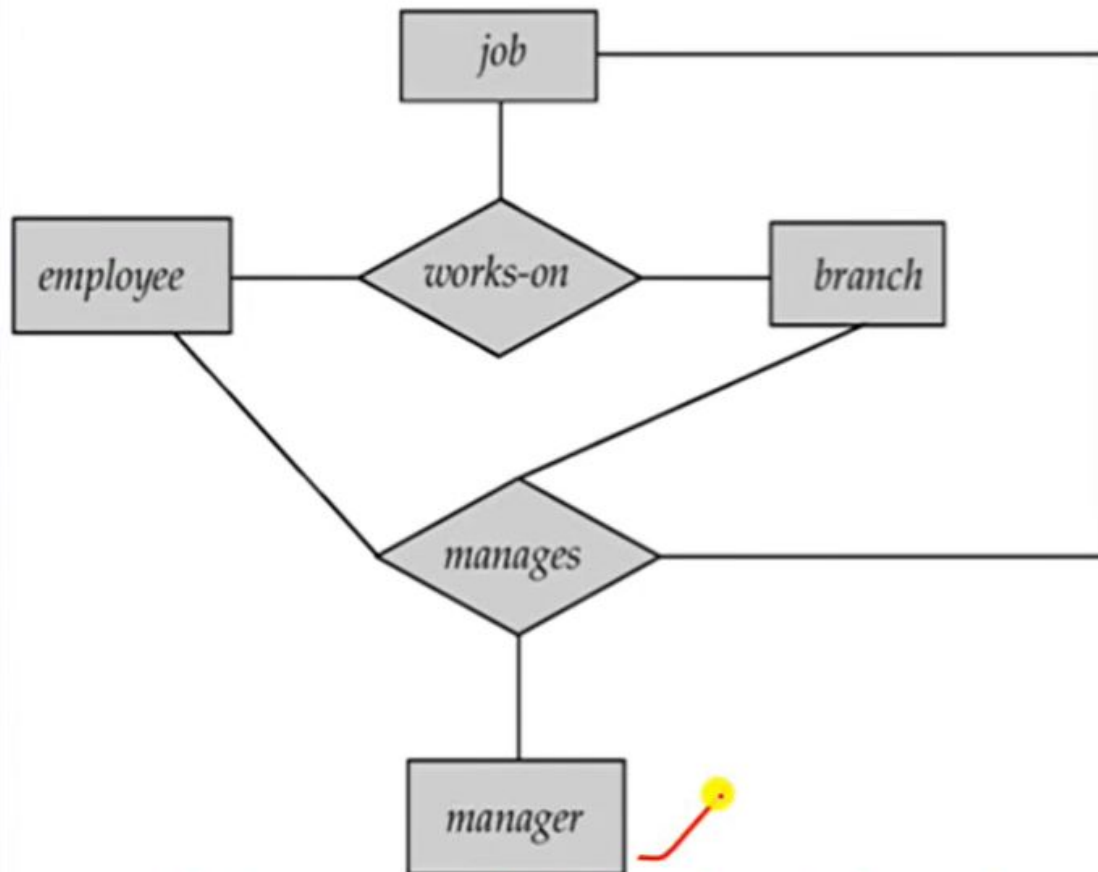
- Basic E-R model can't represent relationships involving other relationships
- Consider a ternary relationship **works_on** between **Employee, Branch and Job**.
 - An **employee works on** a particular **job** at a particular **branch**



- Suppose we want to assign a **manager** for jobs performed by an employee at a branch (i.e. want to assign managers to each employee, job, branch combination)
 - Need a separate manager entity-set
 - Relationship between each manager, employee, branch, and job entity



Example: (Redundant Relationship)



❑ Relationship sets ***works-on*** and ***manages*** represent overlapping (redundant) information

- Every *manages* relationship corresponds to a *works-on* relationship
- However, some *works-on* relationships may not correspond to any *manages* relationships
 - So we can't discard the *works-on* relationship

ER Diagram with Redundant Relationship



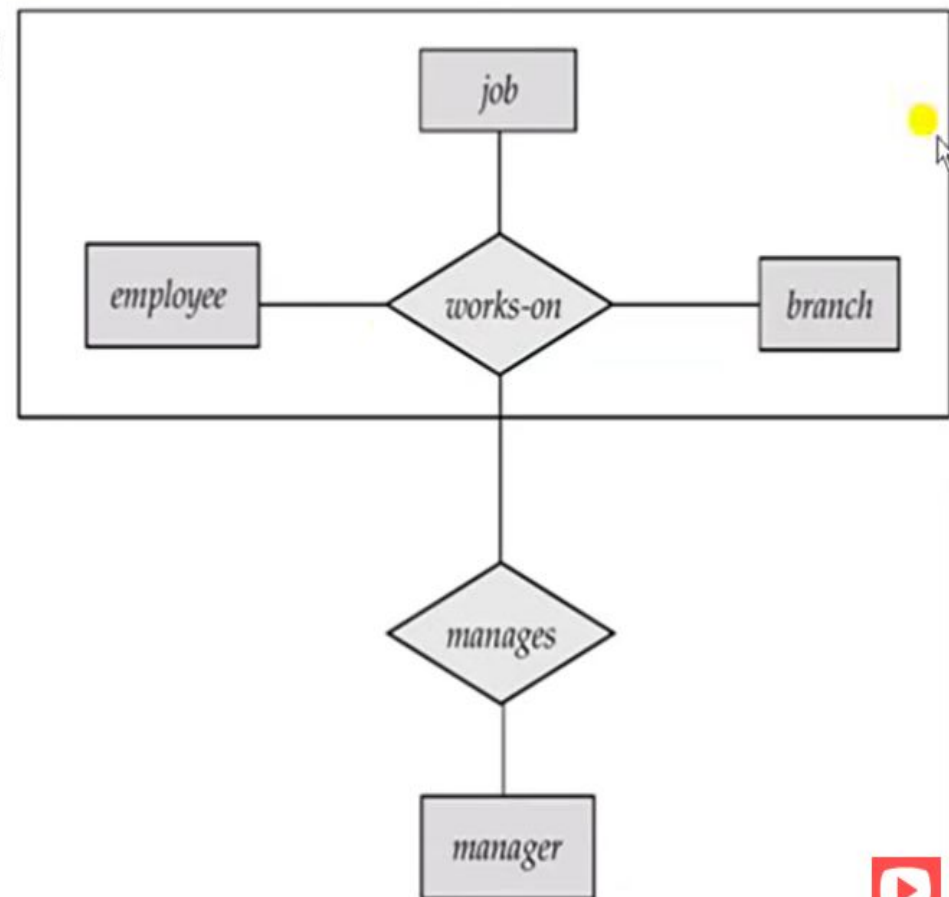
Aggregation

- Eliminate this redundancy via **aggregation**
 - ▣ Treat relationship as an abstract entity
 - ▣ Allows relationships between relationships
 - ▣ Abstraction of relationship into new entity



Example: (Aggregation)

- With **Aggregation** (without introducing redundancy) the ER diagram can be represented as:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager

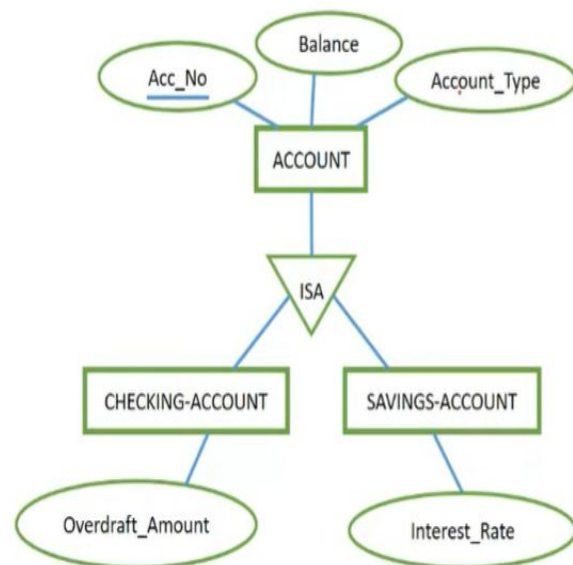


ER Diagram with Aggregation

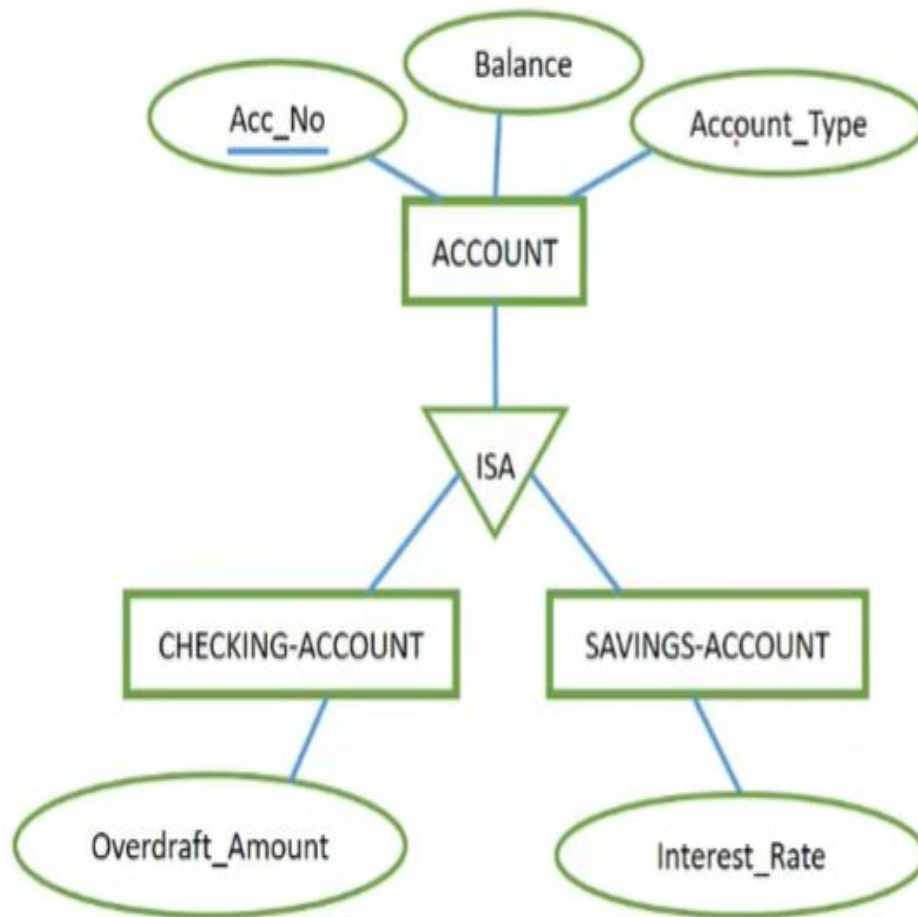
Constraints on Specialization and Generalization

Condition defined : In condition-defined lower-level entity sets, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate. For example, assume that the higher-level entity set Account is having attribute Account_type. Only those entities that satisfy the condition Account_type = "Savings account" are allowed to belong to the lower-level entity set 'Savings_account'. All entities that satisfy the condition Account_type = "checking account" are included in checking account.

Condition - Defined

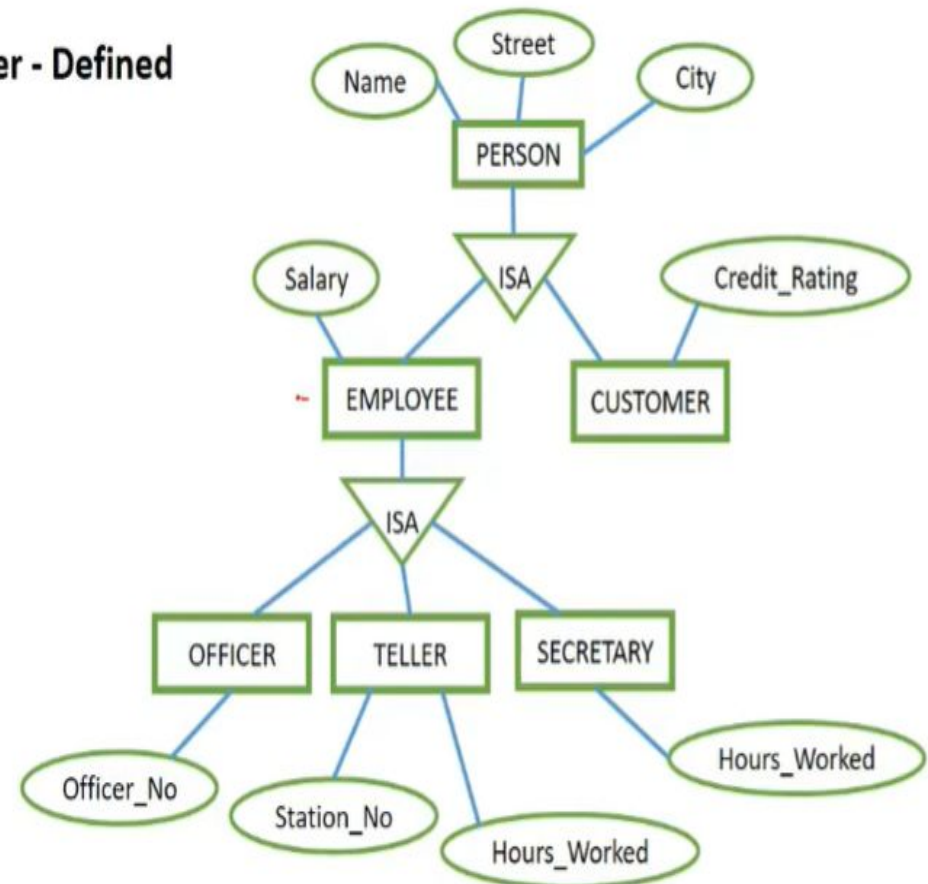


Condition - Defined



Constraints on Specialization and Generalization

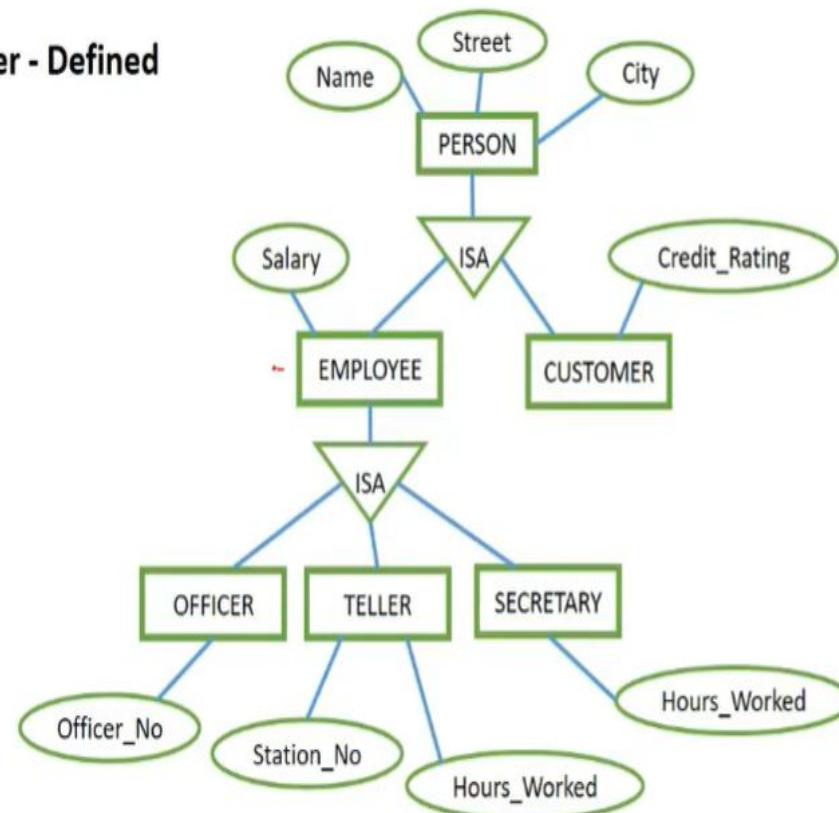
User - Defined



Constraints on Specialization and Generalization

User defined : These types of constraints are defined by user. For example, let us assume that, after 3 months of employment bank employees are assigned to one of four work teams. We therefore represent the teams as four lower-level entity sets of the higher-level Employee entity set. A given Employee is assigned to one of the four teams by incharge of the teams.

User - Defined

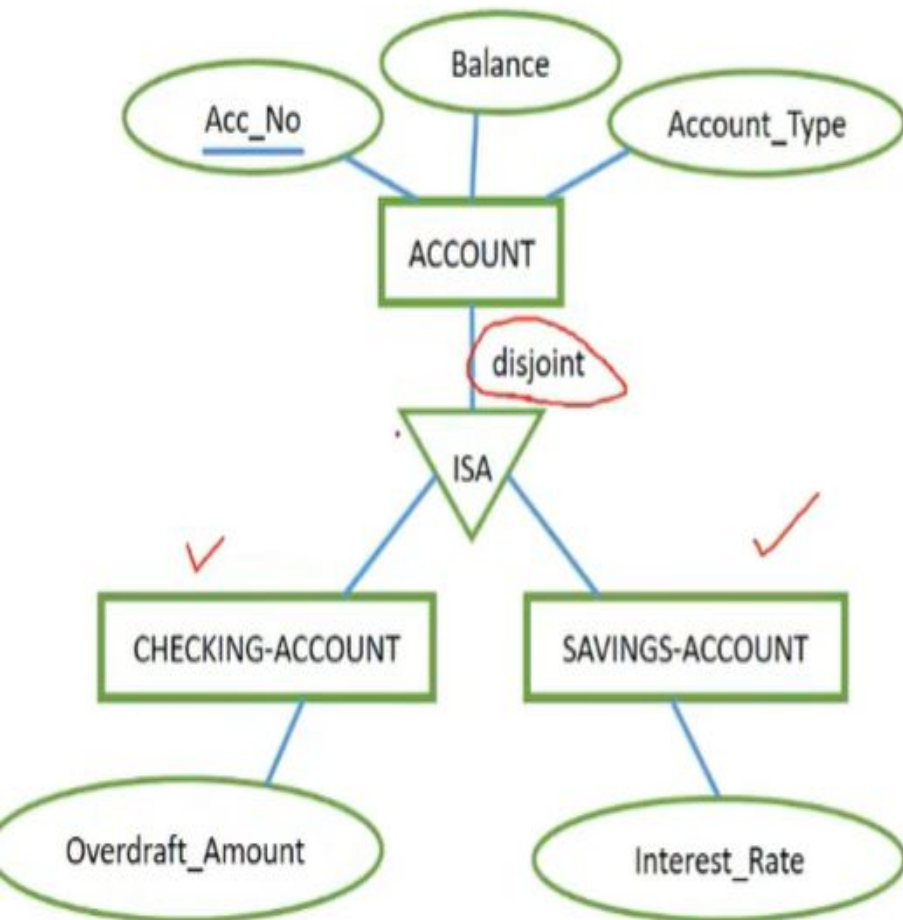


Constraints on Specialization and Generalization

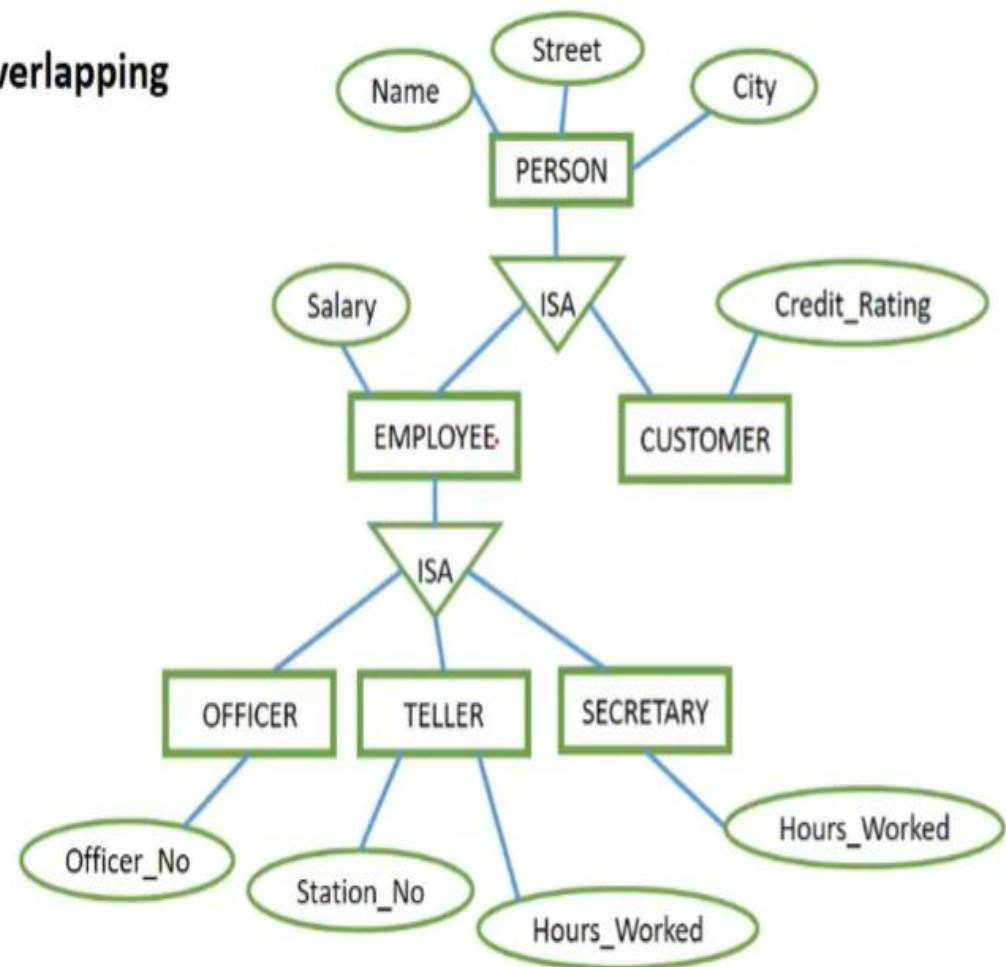
Disjoint : A *disjointness* constraint requires that an entity belong to only one lower-level entity set. For example, an Account entity may be either Saving_account or Checking_account. It satisfies just one condition at a time.

Overlapping : In overlapping generalization, the same entity may belong to more than one lower-level entity set within a single generalization.

Disjoint Constraint



Overlapping



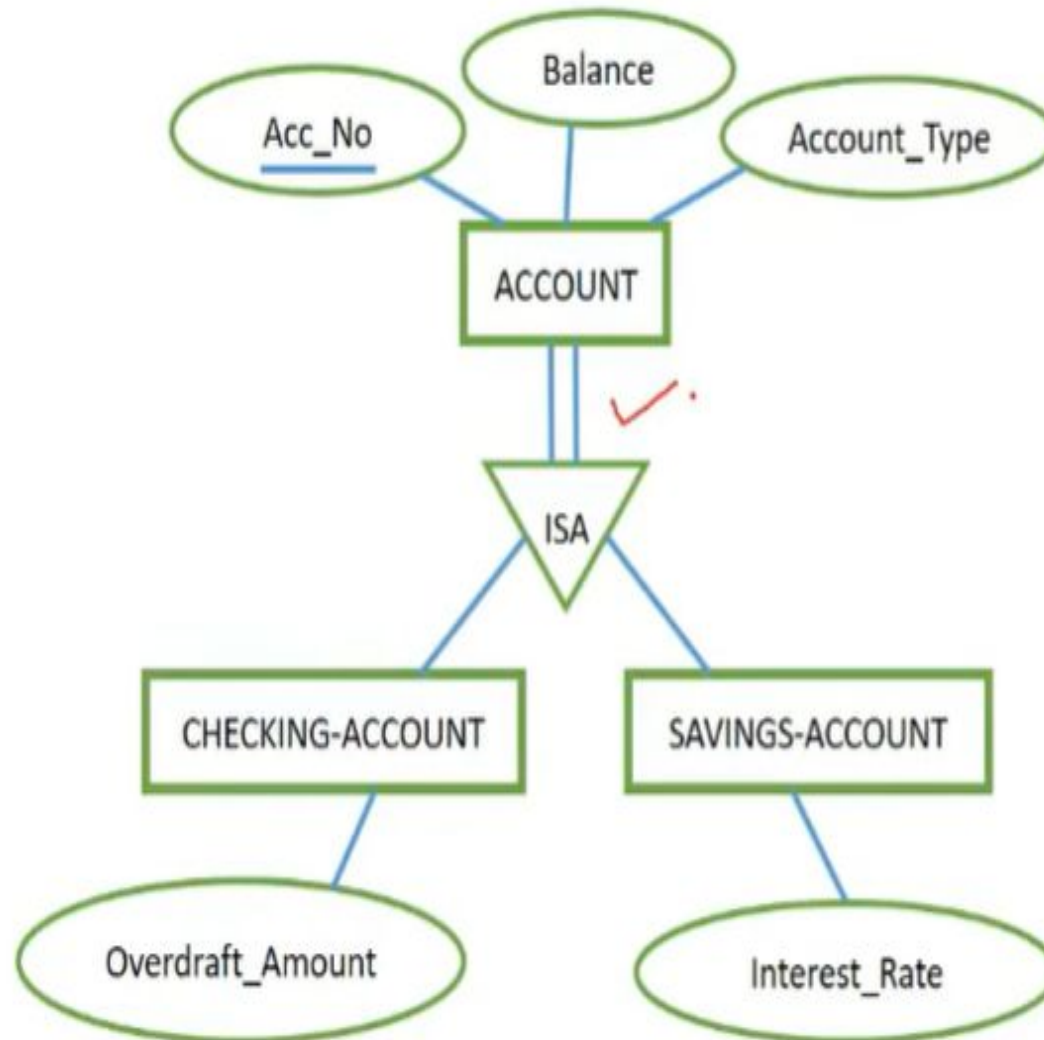
Constraints on Specialization and Generalization

A final constraint, is a completeness constraint on a generalization/specialization, which specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within the generalization/specialization. This constraint may be one of the following :

- **Total generalization or specialization** : Each higher-level entity must belong to a lower-level entity set.
- **Partial generalization or specialization** : Some higher-level entities may not belong to any lower-level entity set. Partial generalization is the default.

Constraints on Specialization and Generalization

Total Specialization/ Generalization



Constraints on Specialization and Generalization

can specify total generalization in an E-R diagram by using a double line to connect the box representing the higher-level entity set to the triangle symbol.

For example : Employees are assigned to a team only after 3 months on the job. Some Employee entities may not be members of any of the lower-level team entity sets. We may characterize the team entity sets more fully as a partial, overlapping specialization of Employee.

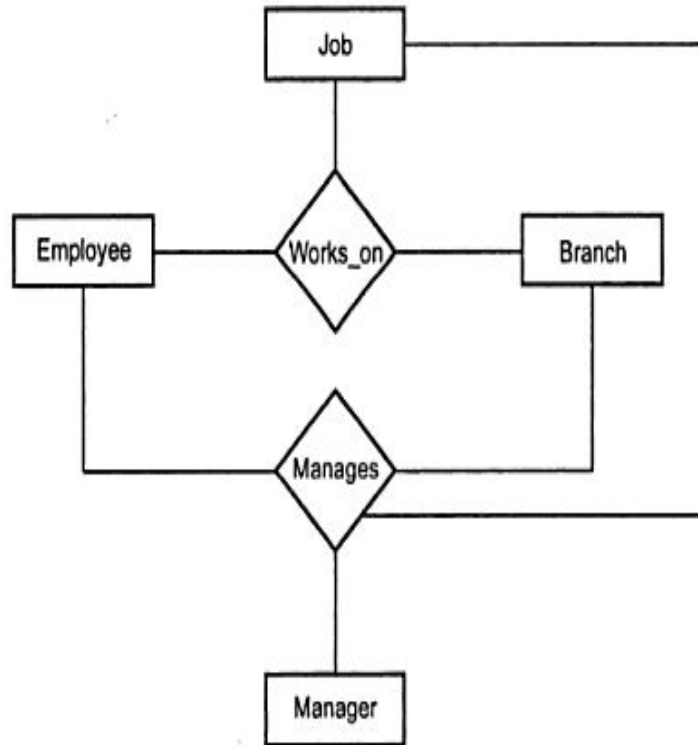
The generalization of Checking_account and Savings_account into Account is a total disjoint generalization.

Attribute Inheritance

“A crucial property of the higher and lower-level entities created by specialization and generalization is **attribute inheritance**”.

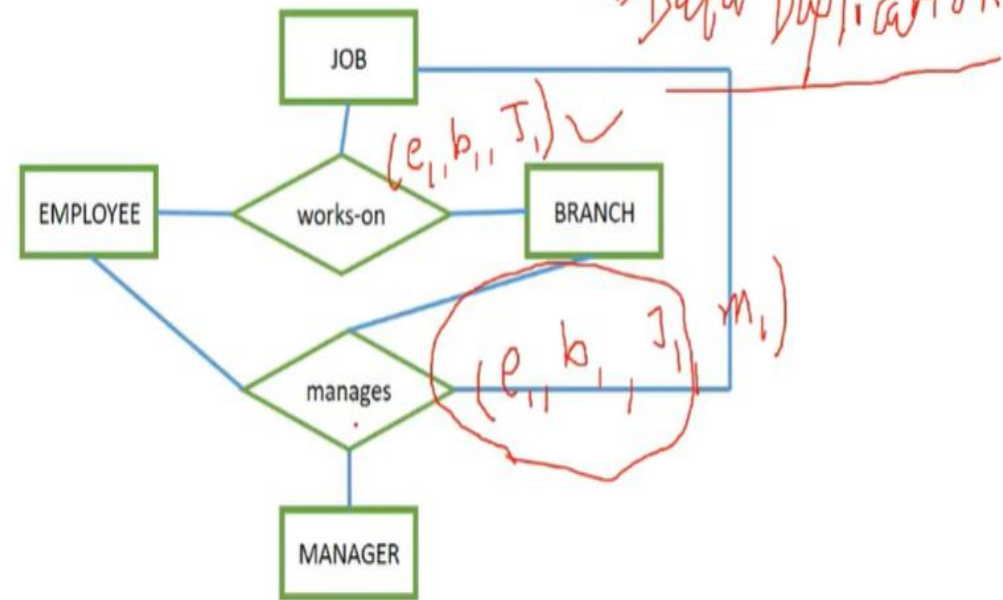
The attributes of the higher-level entity sets are said to be inherited by the lower-level entity sets. For example, Customer and Employee inherit the attributes of the Person. Thus, Customer is described by Name, Street, City and with additional attribute Customer_id. Similarly, Employee is described by Name, Street, City and additional attributes Employee_code and Salary.

Aggregation

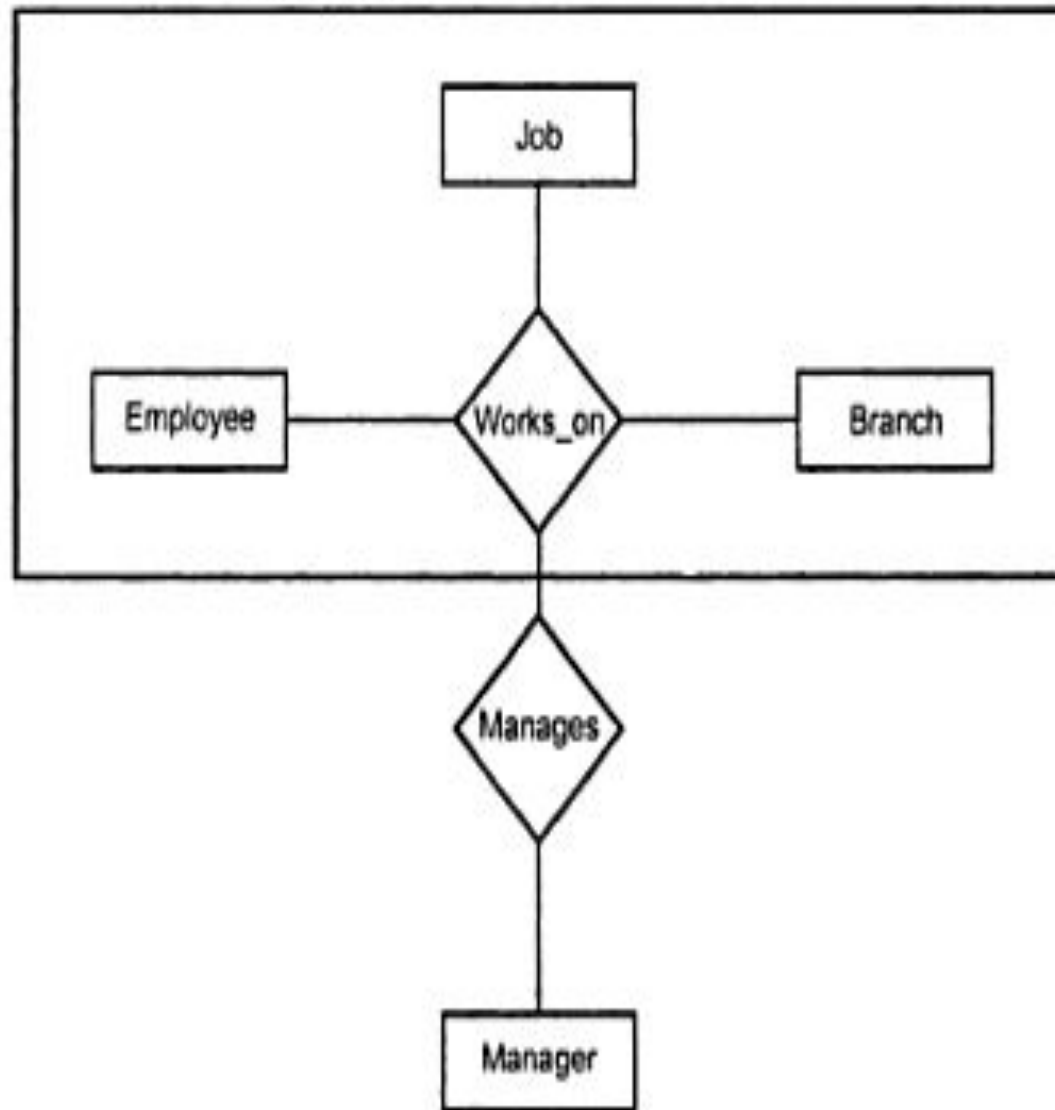


E-R diagram with redundant relationships

ER Diagram with redundant relationships



Aggregation



E-R diagram with aggregation

Aggregation

The best way to model above situation is to use aggregation. Aggregation is an abstraction through which relationships are treated as higher-level entities. Thus, the relationship set Works_on relating the entity sets Employee, Branch and Job is considered as a higher_level entity set called Works_on. We can then create a binary relationship manages between Works_on and Manager to represent who manages what tasks.

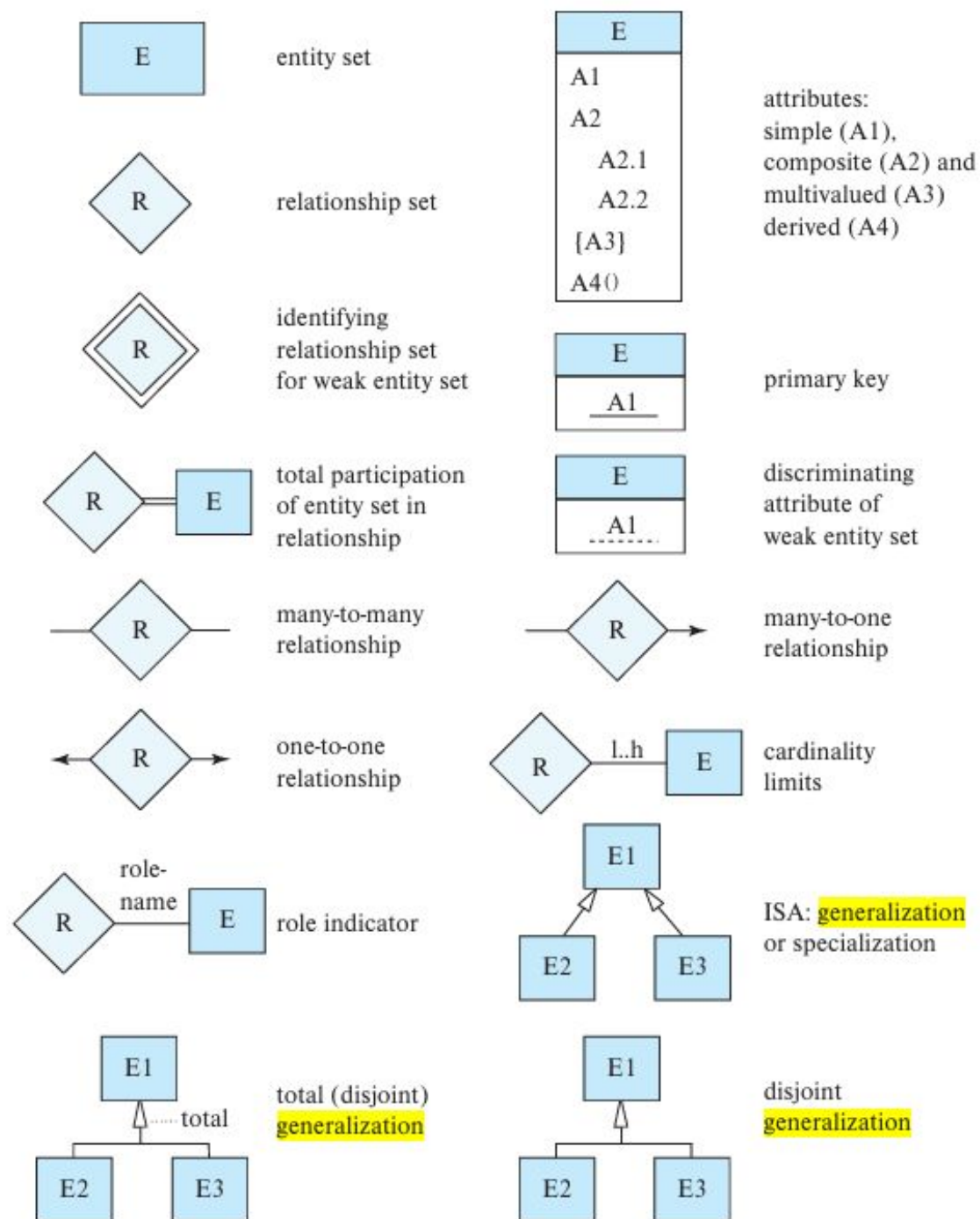


Figure 6.26 Symbols used in the E-R notation.

